

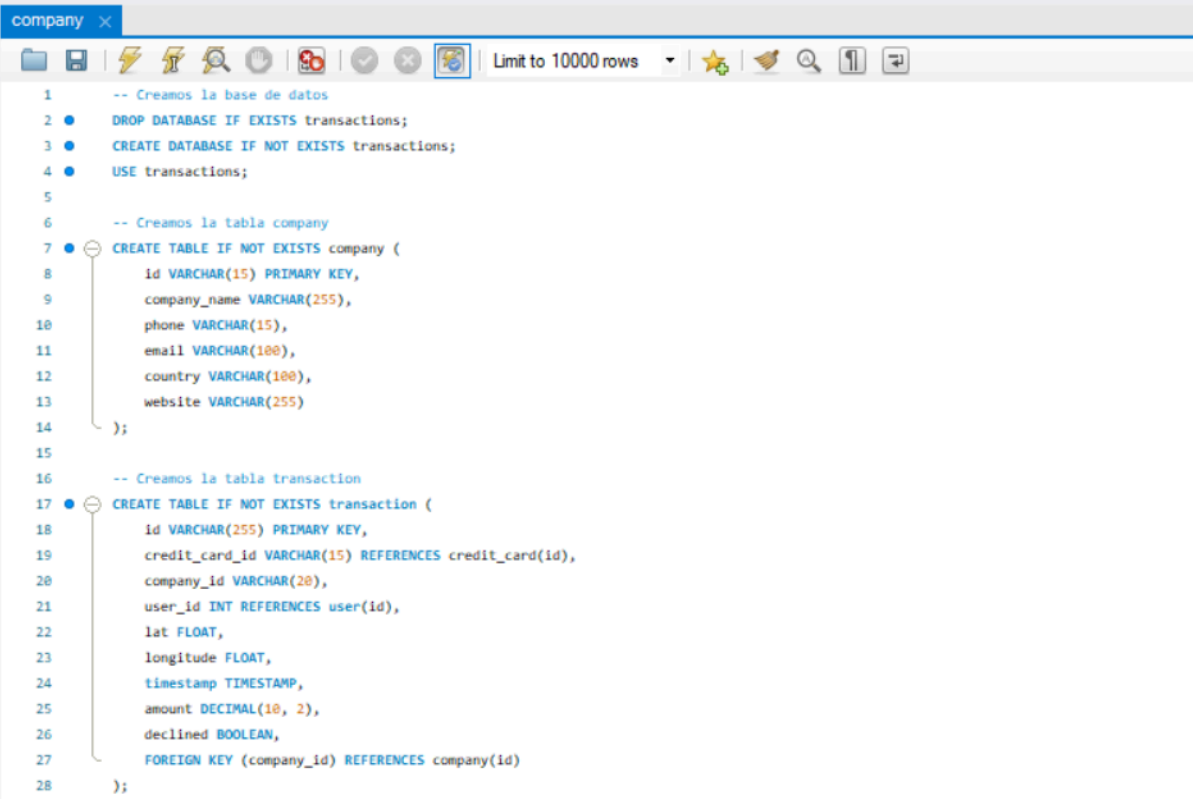
Nivell 1

Exercici 1

A partir dels documents adjunts (estructura_dades i dades_introduir), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.

Respuesta:

Se utiliza los documentos adjuntos estructura_dades y dades_introduir para crear la base de datos y las tablas company y transaction, e insertar todos los datos en ellas, añadiendo al principio DROP DATABASE IF EXISTS.



The screenshot shows a SQL IDE window titled 'company'. The SQL editor contains the following code:

```
1 -- Creamos la base de datos
2 DROP DATABASE IF EXISTS transactions;
3 CREATE DATABASE IF NOT EXISTS transactions;
4 USE transactions;
5
6 -- Creamos la tabla company
7 CREATE TABLE IF NOT EXISTS company (
8     id VARCHAR(15) PRIMARY KEY,
9     company_name VARCHAR(255),
10    phone VARCHAR(15),
11    email VARCHAR(100),
12    country VARCHAR(100),
13    website VARCHAR(255)
14 );
15
16 -- Creamos la tabla transaction
17 CREATE TABLE IF NOT EXISTS transaction (
18     id VARCHAR(255) PRIMARY KEY,
19     credit_card_id VARCHAR(15) REFERENCES credit_card(id),
20     company_id VARCHAR(20),
21     user_id INT REFERENCES user(id),
22     lat FLOAT,
23     longitude FLOAT,
24     timestamp TIMESTAMP,
25     amount DECIMAL(10, 2),
26     declined BOOLEAN,
27     FOREIGN KEY (company_id) REFERENCES company(id)
28 );
```

Below the editor is the 'Output' panel, which shows the execution results of the SQL statements:

#	Time	Action	Message
✓ 1	10:29:31	DROP DATABASE IF EXISTS transactions	3 row(s) affected
✓ 2	10:29:31	CREATE DATABASE IF NOT EXISTS transactions	1 row(s) affected
✓ 3	10:29:31	USE transactions	0 row(s) affected
✓ 4	10:29:31	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(...	0 row(s) affected
✓ 5	10:29:31	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, credit_card_id VARCHA...	0 row(s) affected

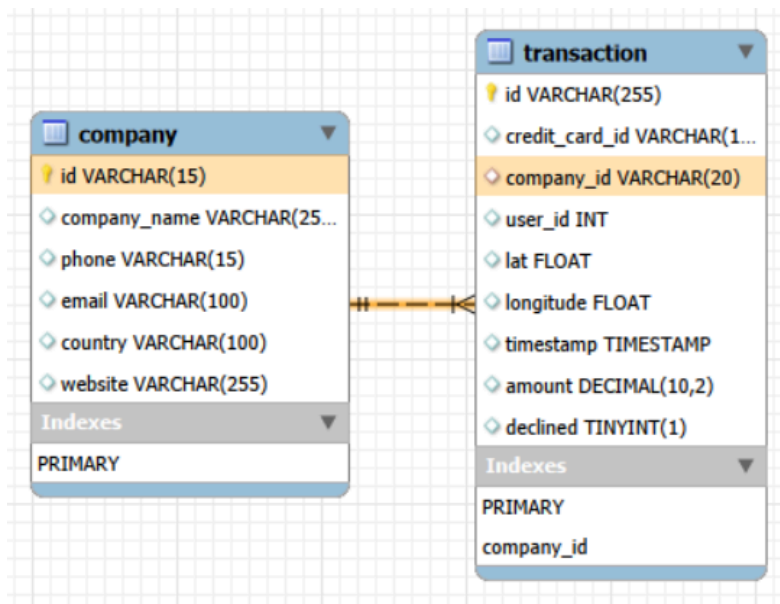
```

Query 1
1 USE transactions;
2
3 -- Insertamos datos de company
4 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
5 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
6 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
7 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
8 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
9 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (

Output
# Time Action Message
100092 19:01:13 INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES (F6665E6B-E543-4E0D-AE2B-... 1 row(s) affected
100093 19:01:13 INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES (5CDDCB22-1ECB-43E5-8538-... 1 row(s) affected
100094 19:01:13 INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES (8007626-1186-4973-BBA-... 1 row(s) affected
100095 19:01:13 INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES (31FC9ED-96AA-4F1B-9890-... 1 row(s) affected
100096 19:01:13 INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES (EBB26F62-F637-469B-B464-... 1 row(s) affected
100097 19:01:13 INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES (83380605-AE39-48B2-AA25-... 1 row(s) affected
100098 19:01:13 INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES (C35A2438-ECE2-4006-A6EB-... 1 row(s) affected

```

En este esquema nos encontramos dos tablas: company y transaction:



La tabla company se compone de datos de empresas, y cuenta con las variables: (company) id, nombre, teléfono, email, país, y página web.

Por otro lado la tabla transaction contiene los datos de cada transacción realizada por dichas empresas, y se compone de las variables: (transaction) id, tarjeta de crédito, company id (con la que se relaciona con la otra tabla), user id, latitud, longitud, timestamp (hora y fecha exacta), coste de la transacción, y si ha sido rechazado o no.

Por todo ello, la tabla company es una tabla de dimensiones, ya que describe la empresa como tal, mientras que la tabla transaction es una tabla de hechos, ya que es un registro de operaciones puntuales, y tiene valores como el importe de la transacción.

De hecho, una empresa puede realizar varias transacciones, mientras que cada transacción va ligada a una empresa, por lo que la relación será de 1:N (company:transaction).

En cuanto a granularidad, el nivel de detalle de cada tabla es de una fila por empresa en la tabla company, y una fila por transacción en la fila transaction.

Ambas tablas cuentan con un índice Primary que hace referencia al Primary Key de cada una y, además, la tabla transaction cuenta con un índice sobre la variable company_id.

El Primary Key de cada uno es, por tanto, la variable id de cada una, y la tabla transaction cuenta con el Foreign Key company_id, que hace referencia a la variable id de la tabla company, tal y como he comentado anteriormente.

Exercici 2

Utilitzant JOIN realitzaràs les següents consultes:

- Llistat dels països que estan generant vendes.
- Des de quants països es generen les vendes.
- Identifica la companyia amb la mitjana més gran de vendes.

Resposta:

- Llistat dels països que estan generant vendes.

The screenshot shows a database management tool interface. At the top, a tab labeled 'company' is active. Below the toolbar, a SQL query is entered in a text area:

```
1 SELECT c.country
2 FROM transactions.company AS c
3 JOIN transactions.transaction AS t
4 ON c.id = t.company_id
5 WHERE t.declined = 0
6 GROUP BY c.country;
```

Below the query editor, the 'Result Grid' tab is selected, displaying a table with the following data:

country
Germany
Australia
United States
New Zealand
Norway

At the bottom, the 'Output' tab is selected, showing a log of database actions:

#	Time	Action	Message
100104	10:40:35	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, decline...	1 row(s) affected
100105	10:40:35	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, decline...	1 row(s) affected
100106	10:40:35	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, decline...	1 row(s) affected
100107	10:48:11	SELECT * FROM transactions.company LIMIT 0, 10000	100 row(s) returned
100108	10:50:22	SELECT c.country FROM transactions.company AS c JOIN transactions.transaction AS t ON c.id = t.compan...	15 row(s) returned

- Des de quants països es generen les vendes.

company x

Limit to 10000 rows

```

1 • SELECT COUNT(DISTINCT c.country) AS SellerCountries
2 FROM transactions.company AS c
3 JOIN transactions.transaction AS t
4 ON c.id = t.company_id
5 WHERE t.declined = 0;

```

Result Grid

SellerCountries
15

Result 3 x

Output

Action Output

#	Time	Action	Message
✓ 100105	10:40:35	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, decline...	1 row(s) affected
✓ 100106	10:40:35	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, decline...	1 row(s) affected
✓ 100107	10:48:11	SELECT * FROM transactions.company LIMIT 0, 10000	100 row(s) returned
✓ 100108	10:50:22	SELECT c.country FROM transactions.company AS c JOIN transactions.transaction AS t ON c.id = t.compan...	15 row(s) returned
✓ 100109	10:51:14	SELECT COUNT(DISTINCT c.country) AS SellerCountries FROM transactions.company AS c JOIN transactio...	1 row(s) returned

- Identifica la companyia amb la mitjana més gran de vendes.

company x

Limit to 10000 rows

```

1 • SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS avg_amount
2 FROM transactions.company AS c
3 JOIN transactions.transaction AS t
4 ON c.id = t.company_id
5 WHERE t.declined = 0
6 GROUP BY c.id, c.company_name
7 ORDER BY avg_amount DESC
8 LIMIT 1;

```

Result Grid

	id	company_name	avg_amount
▶	b-2222	Ac Fermentum Incorporated	284.91

Result 4 x

Output

Action Output

#	Time	Action	Message
✓ 100106	10:40:35	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, decline...	1 row(s) affected
✓ 100107	10:48:11	SELECT * FROM transactions.company LIMIT 0, 10000	100 row(s) returned
✓ 100108	10:50:22	SELECT c.country FROM transactions.company AS c JOIN transactions.transaction AS t ON c.id = t.compan...	15 row(s) returned
✓ 100109	10:51:14	SELECT COUNT(DISTINCT c.country) AS SellerCountries FROM transactions.company AS c JOIN transactio...	1 row(s) returned
✓ 100110	10:51:50	SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS avg_amount FROM transactions.company AS...	1 row(s) returned

Exercici 3

Utilitzant només subconsultes (sense utilitzar JOIN):

- Mostra totes les transaccions realitzades per empreses d'Alemanya.
- Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.
- Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.

Resposta:

- Mostra totes les transaccions realitzades per empreses d'Alemanya.

The screenshot shows a database interface with a SQL query editor and a results grid. The query is as follows:

```
1 • SELECT *
2 FROM transactions.transaction AS t
3 WHERE t.declined = 0
4 AND EXISTS (
5     SELECT 1
6     FROM transactions.company AS c
7     WHERE c.id = t.company_id
8     AND country = 'Germany'
9 );
```

The results grid displays the following data:

	id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
▶	00138D3B-206D-4C03-94B7-63A2676EB9B4	CcS-4899	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
	0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	CcS-5070	b-2222	489	41.3814	2.18176	2020-12-17 18:15:37	316.90	0
	00201A11-2E62-44C4-941D-198FC8DB77F0	CcU-3512	b-2222	193	55.5704	-3.65129	2021-01-22 23:44:27	453.04	0
	00235618-0A5C-4D49-9DCB-B3A9405D8923	CcS-8137	b-2222	3556	59.8421	18.729	2020-09-09 15:43:19	263.14	0
	005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	CcS-8998	b-2222	4417	41.1591	-8.63905	2024-05-15 09:10:11	442.01	0
	00687139-48B2-4FFA-8E73-B20376F04AB4	CcS-4870	b-2222	289	51.1966	10.4669	2019-03-09 19:37:49	524.84	0
	0074F4DD-32F1-4827-8758-55896314623A	CcS-8081	b-2222	3500	39.7016	-8.50325	2016-12-26 23:06:57	491.90	0
	00AAB9CD-39D6-4DCB-8A1D-13BE73DC90A9	CcS-6797	b-2222	2216	55.7652	-3.76245	2021-04-25 03:06:59	167.15	0

Below the results grid, there is an 'Action Output' table showing the execution of the query and other database actions:

#	Time	Action	Message
✓ 100107	10:48:11	SELECT * FROM transactions.transaction LIMIT 0, 10000	100 row(s) returned
✓ 100108	10:50:22	SELECT c.country FROM transactions.transaction AS t JOIN transactions.transaction AS t ON c.id = t.compan...	15 row(s) returned
✓ 100109	10:51:14	SELECT COUNT(DISTINCT c.country) AS SellerCountries FROM transactions.transaction AS t JOIN transactio...	1 row(s) returned
✓ 100110	10:51:50	SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS avg_amount FROM transactions.transaction AS t...	1 row(s) returned
✓ 100111	11:07:56	SELECT * FROM transactions.transaction AS t WHERE t.declined = 0 AND EXISTS (SELECT 1 FROM tra...	10000 row(s) returned

- Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.

company x

```

1 • SELECT DISTINCT id, company_name
2   FROM transactions.company AS c
3  WHERE EXISTS (
4      SELECT 1
5      FROM transactions.transaction AS t
6      WHERE declined = 0
7      AND c.id = t.company_id
8      AND amount > (
9          SELECT AVG(amount)
10         FROM transactions.transaction
11        WHERE declined = 0
12      )
13  );

```

Limit to 10000 rows

Result Grid

id	company_name
b-2222	Ac Fermentum Incorporated
b-2226	Magna A Neque Industries
b-2230	Fusce Corp.
b-2234	Convallis In Incorporated
b-2238	Ante Iaculis Nec Foundation
b-2242	Donec Ltd
b-2246	Sed Nunc Ltd
b-2250	Amet Nulla Donec Corporation
b-2254	Nascetur Ridiculus Mus Inc.

company 6 x

Output

Action Output

#	Time	Action	Message
✓ 100108	10:50:22	SELECT c.country FROM transactions.company AS c JOIN transactions.transaction AS t ON c.id = t.compan...	15 row(s) returned
✓ 100109	10:51:14	SELECT COUNT(DISTINCT c.country) AS SellerCountries FROM transactions.company AS c JOIN transactio...	1 row(s) returned

- Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.

company x

Limit to 10000 rows

```

1 • DELETE FROM transactions.company AS c
2 WHERE NOT EXISTS (
3     SELECT 1
4     FROM transactions.transaction AS t
5     WHERE c.id = t.company_id
6 );
7 -- No existe ninguna empresa sin transacciones registradas
8 -- así que hacer un delete de éstas (tal y como pide el enunciado)
9 -- no cambia la lista de empresas resultante
10
11 • SELECT * FROM transactions.company;
12 -- Para ver el listado resultante de empresas

```

Result Grid

	id	company_name	phone	email	country	website
▶	b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
	b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
	b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
	b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.couk	Germany	https://cnn.com/user/110
	b-2238	Ante Iaculis Nec Foundation	08 23 04 99 53	sed.dictum.proin@outlook.ca	New Zealand	https://netflix.com/settings
	b-2242	Donec Ltd	01 25 51 37 37	at.iaculis@hotmail.couk	Norway	https://nytimes.com/user/110
	b-2246	Sed Nunc Ltd	02 62 64 73 48	nibh@yahoo.org	United Kingdom	https://cnn.com/one
	b-2250	Amet Nulla Donec Corporation	07 15 25 14 74	mattis.integer.eu@protonmail.net	Italy	https://netflix.com/sub/cars
	b-2254	Nascetur Ridiculus Mus Inc.	06 26 87 61 84	suspendisse.dui@icloud.net	United States	https://ebay.com/sub

company 7 x

Output

Action Output

#	Time	Action	Message
✓ 100110	10:51:50	SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS avg_amount FROM transactions.company AS...	1 row(s) returned
✓ 100111	11:07:56	SELECT * FROM transactions.transaction AS t WHERE t.declined = 0 AND EXISTS (SELECT 1 FROM tra...	10000 row(s) returned

Exercici 4

La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit.

La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company").

Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit".

Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.

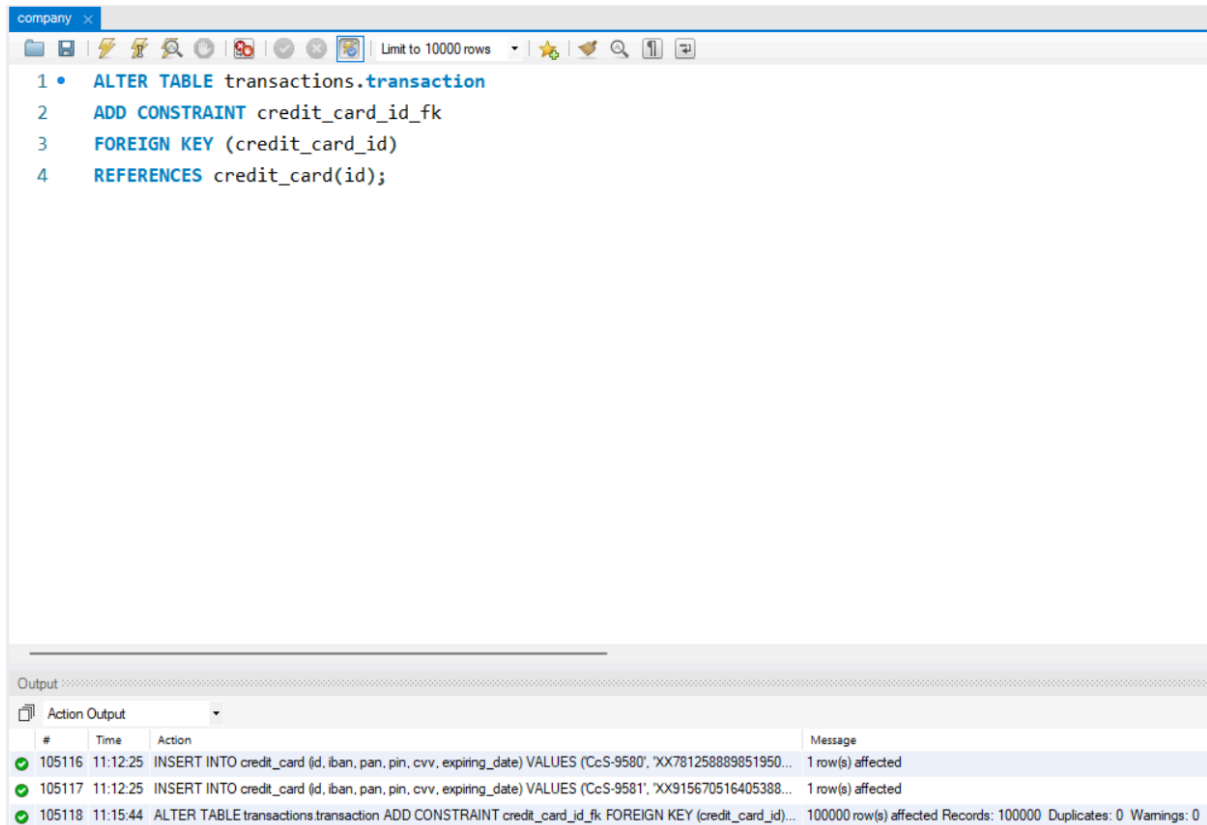
Respuesta:

Creación de tabla e inserción de datos.

```
1 CREATE TABLE IF NOT EXISTS credit_card (  
2     id VARCHAR(50) PRIMARY KEY,  
3     iban VARCHAR(50),  
4     pan VARCHAR(50),  
5     pin VARCHAR(4),  
6     cvv VARCHAR(3),  
7     expiring_date VARCHAR(8)  
8 );  
9  
10 -- Insertamos datos de credit_card  
11 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2938', 'TR301950312213576817638661', '5424465566813633',  
12 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2945', 'D026854763748537475216568689', '5142423821948828',  
13 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2952', 'B645IVQL52710525608255', '4556 453 55 5287', '459  
14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2959', 'CR7242477244335841535', '372461377349375', '3583'  
15 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2966', 'BG72LKTQ15627628377363', '448566 886747 7265', '4  
16 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2973', 'PT87806228135092429456346', '544 58654 54343 384'  
17 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2980', 'DE39241881883086277136', '402400 7145845969', '5e  
18 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2987', 'GE89681434837748781813', '3763 747687 76666', '22  
19 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2994', 'BH62714428368066765294', '344283273252593', '7545  
20 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3001', 'CY49087426654774581266832110', '511722 924833 224  
21 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3008', 'LU507216693616119230', '4485744464433884', '1856'  
22 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3015', 'PS119398216295715968342456821', '3784 662233 1738  
23 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3022', 'GT91695162850556977423121857', '5164 1379 4842 39  
24 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3029', 'AZ62317413982441418123739746', '3429 279566 77631  
25 INSERT INTO credit card (id, iban, pan, pin, cvv, expiring date) VALUES ('CcU-3036', 'AZ39336002925842865843941994', '3768 451556 48766
```

#	Time	Action	Message
105113	11:12:25	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9577', 'XX158914078594808...	1 row(s) affected
105114	11:12:25	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9578', 'XX991539646456110...	1 row(s) affected

Alteración de la tabla para definir el Primary Key, y su constraint con la variable credit_card_id de la tabla transaction, a la que he llamado credit_card_id_fk.



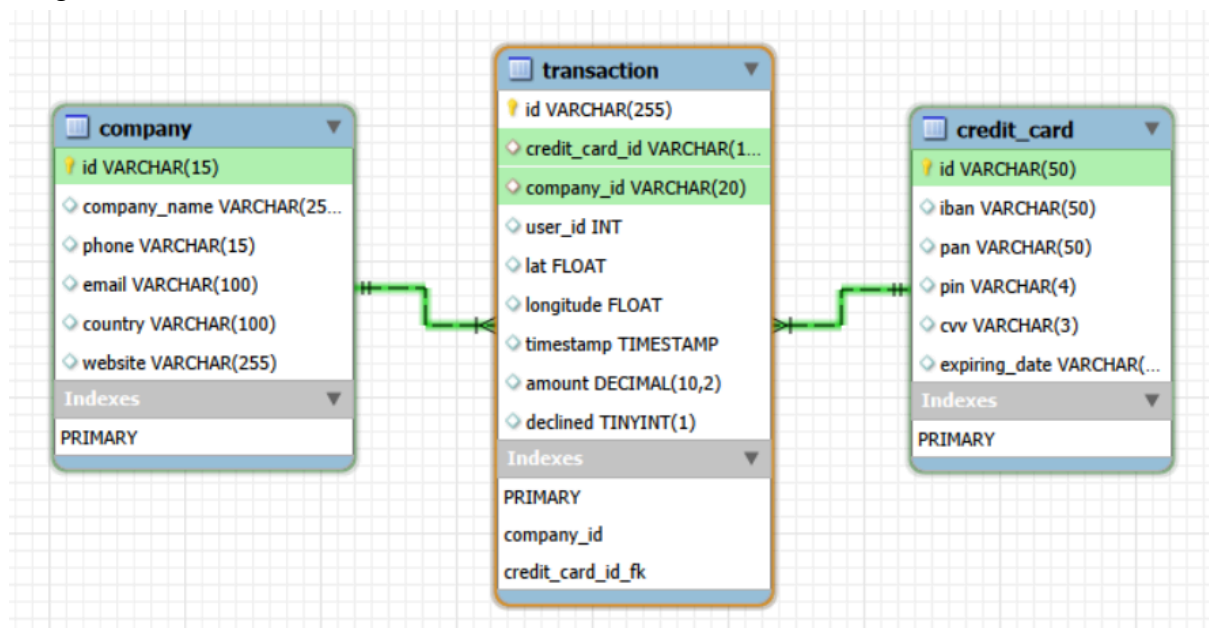
The screenshot shows a database management interface with a tab labeled 'company'. The SQL editor contains the following commands:

```
1 • ALTER TABLE transactions.transaction
2 ADD CONSTRAINT credit_card_id_fk
3 FOREIGN KEY (credit_card_id)
4 REFERENCES credit_card(id);
```

The 'Output' section at the bottom displays the results of the executed actions in a table:

#	Time	Action	Message
105116	11:12:25	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9580', 'XX781258889851950...	1 row(s) affected
105117	11:12:25	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX915670516405388...	1 row(s) affected
105118	11:15:44	ALTER TABLE transactions.transaction ADD CONSTRAINT credit_card_id_fk FOREIGN KEY (credit_card_id)...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Diagrama resultante.



Añadimos una tabla más al esquema anterior.

En este caso la tabla `credit_card` se compone de datos de las tarjetas de crédito utilizadas para realizar las transacciones de la tabla `transaction`. Ésta cuenta con las variables `id`, `iban`, `pan`, `pin`, `cvv`, y fecha de vencimiento.

La tabla `credit_card` es una tabla de dimensiones, ya que tiene variables descriptivas de cada tarjeta de crédito. Por lo tanto, el sistema que se compone de estas tablas es un modelo en estrella, con dos tablas de dimensiones que se relacionan directamente con la tabla de hechos central.

Tal como ocurre con la tabla `company`, la relación entre las tablas `credit_card` y `transaction` es de 1:N (`credit_card:transaction`), ya que una tarjeta de crédito puede realizar varias transacciones, mientras que cada transacción va ligada a una sola tarjeta de crédito.

En cuanto a granularidad, el nivel de detalle de cada tabla es de una fila por cada tarjeta de crédito.

La nueva tabla cuenta con un índice Primary que hace referencia al Primary Key, y la tabla `transaction` añade un nuevo índice sobre la variable `credit_card_id`.

Por lo tanto, la tabla `transaction` usa su variable `credit_card_id` como Foreign Key, que hace referencia a la variable `id` de la tabla `credit_card`.

Exercici 5

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938.

La informació que ha de mostrar-se per a aquest registre és:

TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.

Respuesta:

Antes del cambio, con iban TR301950312213576817638661.

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	id	iban	pan	pin	cvv	expiring_date
▶	CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	10/30/22
✱	NULL	NULL	NULL	NULL	NULL	NULL

Código usado para modificar la variable iban y su resultado.

company

Limit to 10000 rows

1 • UPDATE transactions.credit_card
2 SET iban = 'TR323456312213576817699999'
3 WHERE id = 'CcU-2938';
4
5 • SELECT * FROM transactions.credit_card
6 WHERE id = 'CcU-2938';
7 -- Para mostrar que el cambio se ha realizado

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	10/30/22
NULL	NULL	NULL	NULL	NULL	NULL

credit_card 2

Output

Action Output

#	Time	Action	Message
5008	20:10:24	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9578', 'XX991539646456110567870254', '8999808823061411', '2872...	1 row(s) affected
5009	20:10:24	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9579', 'XX296393091587170202131236', '9690060468678689', '8379...	1 row(s) affected
5010	20:10:24	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9580', 'XX781258889851950806677358', '5541182364498931', '9273...	1 row(s) affected
5011	20:10:24	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX915670516405388124398147', '2624305470167630', '4336...	1 row(s) affected
5012	20:11:13	ALTER TABLE transactions.transaction ADD CONSTRAINT credit_card_id_fk FOREIGN KEY (credit_card_id) REFERENCES credit_card(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
5013	20:12:09	UPDATE transactions.credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2938'	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
5014	20:12:09	SELECT * FROM transactions.credit_card WHERE id = 'CcU-2938' LIMIT 0, 10000	1 row(s) returned

Exercici 6

En la taula "transaction" ingressa una nova transacció amb la següent informació:

Id	108B1D1D-5B23-A76C-55EF-C568E49A99DD
credit_card_id	CcU-9999
company_id	b-9999
user_id	9999
lat	829.999
longitude	-117.999
amount	111.11
declined	0

Respuesta:

Dado que no existe previamente ni una empresa con el id b-9999, ni una tarjeta de crédito con id CcU-9999, se deben crear ambas para poder insertar esta nueva transacción posteriormente.

Como no se proporciona el resto de información para crear correctamente la empresa y la tarjeta de crédito, se creará cada una solamente con el id que si se ha proporcionado, dejando el resto de variables como NULL.

Una vez creada la empresa y tarjeta de crédito necesarias, se crea la transacción con los datos proporcionados.

company

Limit to 10000 rows

```
1 -- Crea la empresa con id b-9999, dejando el resto de variables como NULL
2 • INSERT INTO transactions.company (id) VALUES ('b-9999');
3
4 -- Crea la tarjeta de crédito con id CcU-9999, dejando el resto de variables como NULL
5 INSERT INTO transactions.credit_card (id) VALUES ('CcU-9999');
6
7 -- Crea la transacción ahora que ya existen la empresa y tarjeta de crédito correspondiente
8 • INSERT INTO transactions.transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
9 VALUES ('10881D1D-5B23-A76C-55EF-C568E49A990D', 'CcU-9999', 'b-9999', 9999, 829.999, -117.999, 111.11, 0);
```

Output

Action Output

#	Time	Action	Message
5011	20:10:24	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX915670516405388124398147', '2624305470167630', '4336...	1 row(s) affected
5012	20:11:13	ALTER TABLE transactions.transaction ADD CONSTRAINT credit_card_id_fk FOREIGN KEY (credit_card_id) REFERENCES credit_card(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
5013	20:12:09	UPDATE transactions.credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2938'	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
5014	20:12:09	SELECT * FROM transactions.credit_card WHERE id = 'CcU-2938' LIMIT 0, 10000	1 row(s) returned
5015	20:14:11	INSERT INTO transactions.company (id) VALUES ('b-9999')	1 row(s) affected
5016	20:14:11	INSERT INTO transactions.credit_card (id) VALUES ('CcU-9999')	1 row(s) affected
5017	20:14:11	INSERT INTO transactions.transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ('10881D1D-5B23-A76C-55...	1 row(s) affected

Exercici 7

Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recorda mostrar el canvi realitzat.

Resposta:

The screenshot shows a database management interface with a SQL editor and a results pane.

SQL Editor:

```
1 • ALTER TABLE transactions.credit_card
2 • DROP COLUMN pan;
3
4 • SELECT * FROM transactions.credit_card;
```

Result Grid:

id	iban	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	1819	467	09/27/25
CcS-4858	XX8581768137002436094025	3964	817	12/28/28
CcS-4859	XX7826930491423553609370	4983	277	11/26/26
CcS-4860	XX5559590368835304645299	6876	661	07/27/27
CcS-4861	XX2035182877195191627307	5710	398	04/25/26
CcS-4862	XX4774721462463645409758	4042	174	11/27/26
CcS-4863	XX1476829664245046207111	5969	449	12/27/29
CcS-4864	XX8380298893385731196159	8481	139	02/28/26
CcS-4865	XX7085078596101025280599	7847	903	11/25/28
CcS-4866	XX4792859188206596406839	9271	961	02/28/25
CcS-4867	XX6038298816319374853717	4820	862	11/30/25
CcS-4868	XX3929101617300533068044	6860	549	08/25/25
CcS-4869	XX1566712096111531886465	4775	448	09/29/25
CcS-4870	XX6443663804167732133949	5872	715	01/25/25
CcS-4871	XX3520209528376786066563	7992	643	07/26/26
CcS-4872	XX4091621143155817546334	3642	191	12/29/26

credit_card 3

Output:

#	Time	Action	Message
5013	20:12:09	UPDATE transactions.credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2938'	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
5014	20:12:09	SELECT * FROM transactions.credit_card WHERE id = 'CcU-2938' LIMIT 0, 10000	1 row(s) returned
5015	20:14:11	INSERT INTO transactions.company (id) VALUES (b-9999)	1 row(s) affected
5016	20:14:11	INSERT INTO transactions.credit_card (id) VALUES (CcU-9999)	1 row(s) affected
5017	20:14:11	INSERT INTO transactions.transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ('108B1D1D-5B23-A76C-55...	1 row(s) affected
5018	20:15:11	ALTER TABLE transactions.credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
5019	20:15:11	SELECT * FROM transactions.credit_card LIMIT 0, 10000	5001 row(s) returned

Exercici 8

Descarrega els arxius CSV que trobaràs a l'apartat de recursos:

american_users, european_users, companies, credit_cards, transactions

Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

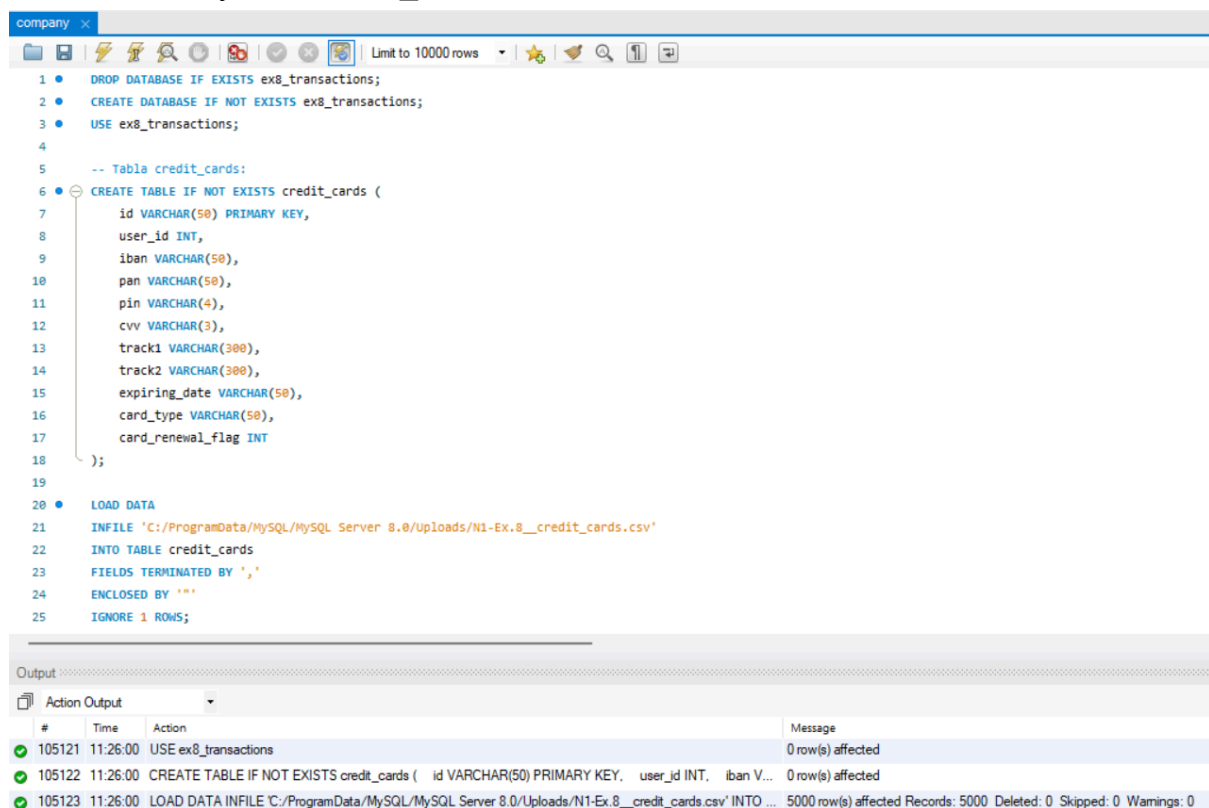
(La taula de products.csv l'utilitzarem més endavant.)

Respuesta:

Primero se crea una nueva base de datos (en este caso llamada ex8_transactions), y una tabla para cada archivo .csv que se va a importar, en la que se establece la Primary Key.

Seguidamente, se carga el .csv desde la carpeta Uploads y se indica el formato para que los valores se ajusten al lenguaje SQL. Se repite este proceso para los archivos .csv, creando una tabla para cada uno:

Base de datos y tabla credit_cards:

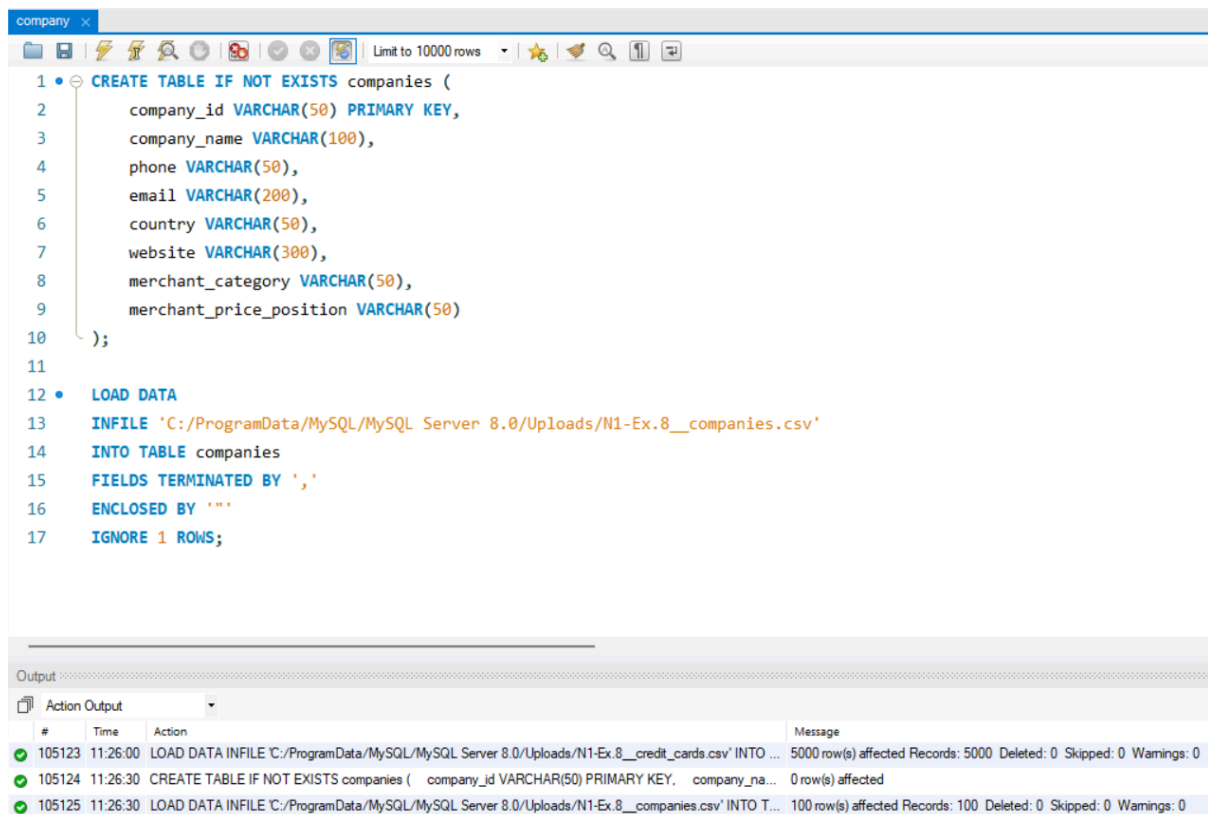


```
company x
1 DROP DATABASE IF EXISTS ex8_transactions;
2 CREATE DATABASE IF NOT EXISTS ex8_transactions;
3 USE ex8_transactions;
4
5 -- Tabla credit_cards:
6 CREATE TABLE IF NOT EXISTS credit_cards (
7   id VARCHAR(50) PRIMARY KEY,
8   user_id INT,
9   iban VARCHAR(50),
10  pan VARCHAR(50),
11  pin VARCHAR(4),
12  cvv VARCHAR(3),
13  track1 VARCHAR(300),
14  track2 VARCHAR(300),
15  expiring_date VARCHAR(50),
16  card_type VARCHAR(50),
17  card_renewal_flag INT
18 );
19
20 LOAD DATA
21 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/uploads/N1-Ex.8__credit_cards.csv'
22 INTO TABLE credit_cards
23 FIELDS TERMINATED BY ','
24 ENCLOSED BY '"'
25 IGNORE 1 ROWS;
```

Output

#	Time	Action	Message
105121	11:26:00	USE ex8_transactions	0 row(s) affected
105122	11:26:00	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(50) PRIMARY KEY, user_id INT, iban V...	0 row(s) affected
105123	11:26:00	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/uploads/N1-Ex.8__credit_cards.csv' INTO ...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0

Tabla companies:

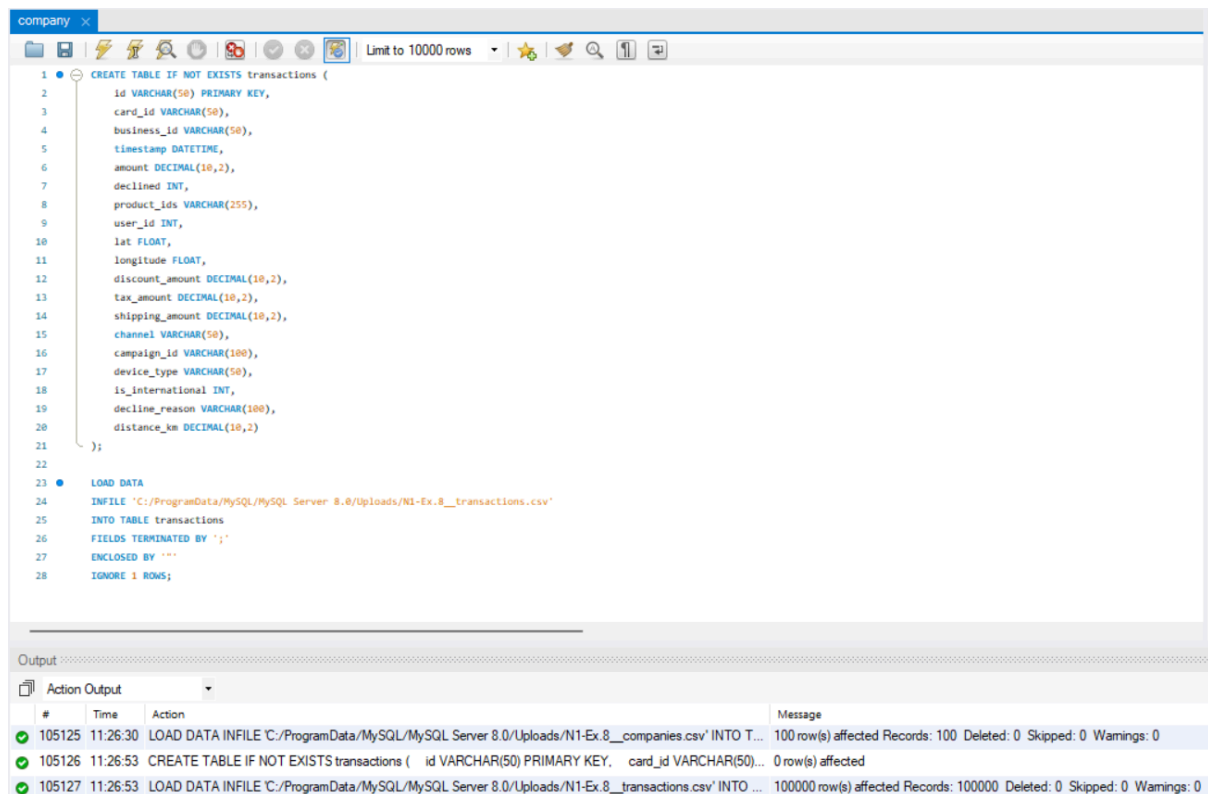


```
1 CREATE TABLE IF NOT EXISTS companies (  
2     company_id VARCHAR(50) PRIMARY KEY,  
3     company_name VARCHAR(100),  
4     phone VARCHAR(50),  
5     email VARCHAR(200),  
6     country VARCHAR(50),  
7     website VARCHAR(300),  
8     merchant_category VARCHAR(50),  
9     merchant_price_position VARCHAR(50)  
10 );  
11  
12 • LOAD DATA  
13 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__companies.csv'  
14 INTO TABLE companies  
15 FIELDS TERMINATED BY ','  
16 ENCLOSED BY ''''  
17 IGNORE 1 ROWS;
```

Output

#	Time	Action	Message
105123	11:26:00	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__credit_cards.csv INTO ...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0
105124	11:26:30	CREATE TABLE IF NOT EXISTS companies (company_id VARCHAR(50) PRIMARY KEY, company_na...	0 row(s) affected
105125	11:26:30	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__companies.csv INTO T...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Tabla transactions:

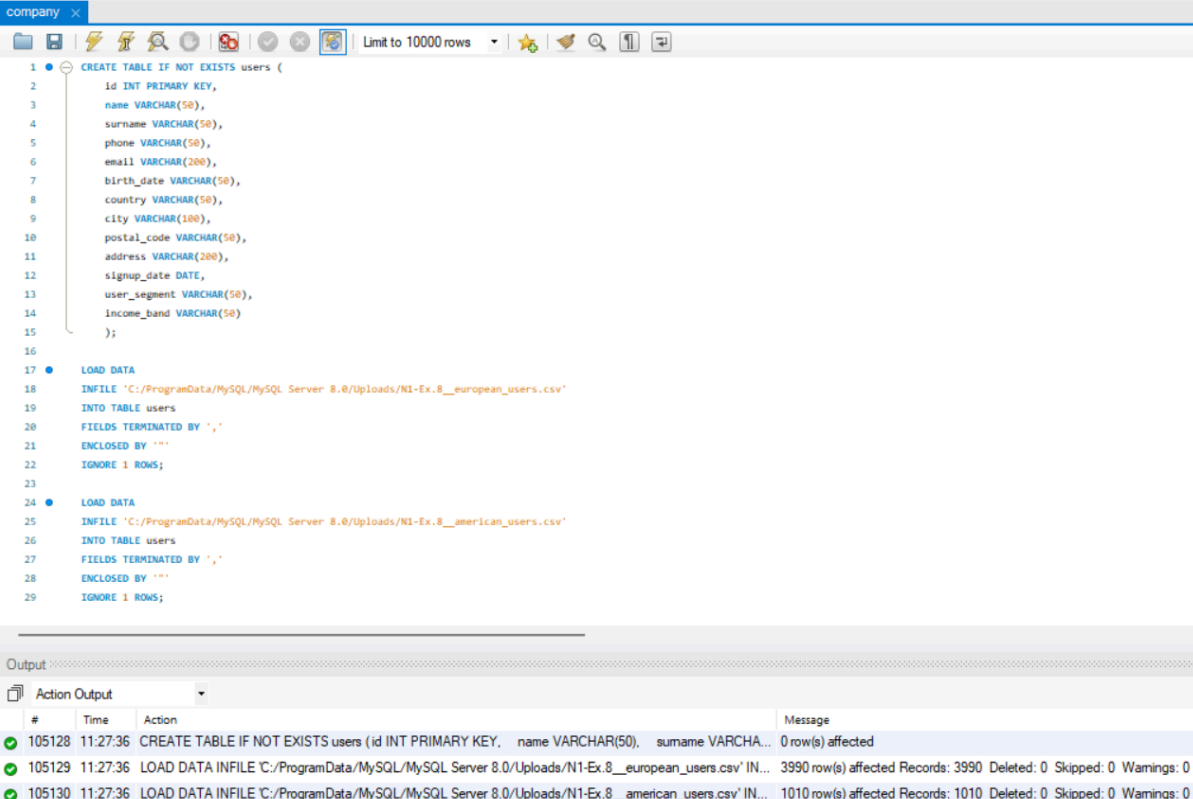


```
1 CREATE TABLE IF NOT EXISTS transactions (  
2     id VARCHAR(50) PRIMARY KEY,  
3     card_id VARCHAR(50),  
4     business_id VARCHAR(50),  
5     timestamp DATETIME,  
6     amount DECIMAL(10,2),  
7     declined INT,  
8     product_ids VARCHAR(255),  
9     user_id INT,  
10    lat FLOAT,  
11    longitude FLOAT,  
12    discount_amount DECIMAL(10,2),  
13    tax_amount DECIMAL(10,2),  
14    shipping_amount DECIMAL(10,2),  
15    channel VARCHAR(50),  
16    campaign_id VARCHAR(100),  
17    device_type VARCHAR(50),  
18    is_international INT,  
19    decline_reason VARCHAR(100),  
20    distance_km DECIMAL(10,2)  
21 );  
22  
23 • LOAD DATA  
24 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv'  
25 INTO TABLE transactions  
26 FIELDS TERMINATED BY ','  
27 ENCLOSED BY ''''  
28 IGNORE 1 ROWS;
```

Output

#	Time	Action	Message
105125	11:26:30	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__companies.csv INTO T...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0
105126	11:26:53	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(50) PRIMARY KEY, card_id VARCHAR(50)...	0 row(s) affected
105127	11:26:53	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv INTO ...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0

Para poder crear el constraint de transactions tanto a american_users como a european_users, al usar el mismo Foreign Key, se crea una tabla users y se cargan los datos de ambos .csv european_users y american_users en ella:



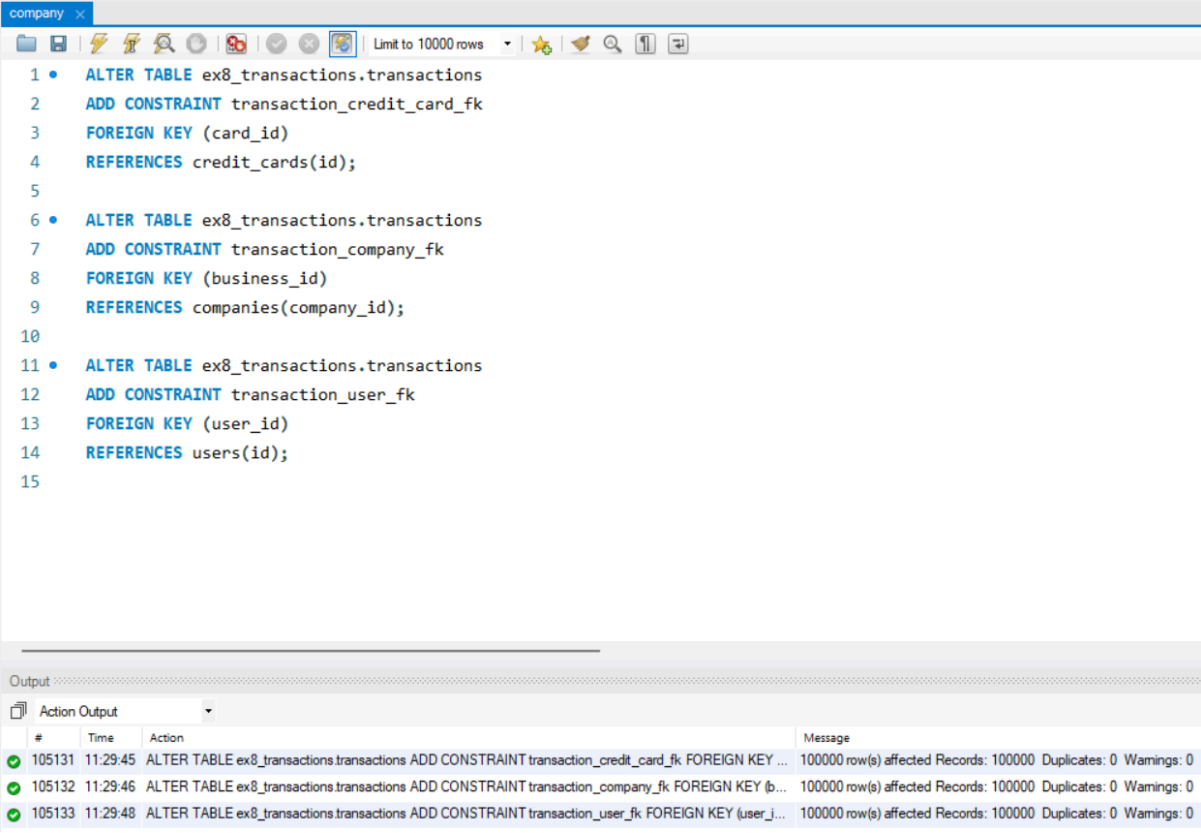
The screenshot displays the MySQL Workbench interface. The top toolbar includes icons for file operations, execution, and navigation. The main editor window contains the following SQL script:

```
1 CREATE TABLE IF NOT EXISTS users (  
2     id INT PRIMARY KEY,  
3     name VARCHAR(50),  
4     surname VARCHAR(50),  
5     phone VARCHAR(50),  
6     email VARCHAR(200),  
7     birth_date VARCHAR(50),  
8     country VARCHAR(50),  
9     city VARCHAR(100),  
10    postal_code VARCHAR(50),  
11    address VARCHAR(200),  
12    signup_date DATE,  
13    user_segment VARCHAR(50),  
14    income_band VARCHAR(50)  
15 );  
16  
17  
18 LOAD DATA  
19 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__european_users.csv'  
20 INTO TABLE users  
21 FIELDS TERMINATED BY ','  
22 ENCLOSED BY ''''  
23 IGNORE 1 ROWS;  
24  
25 LOAD DATA  
26 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_users.csv'  
27 INTO TABLE users  
28 FIELDS TERMINATED BY ','  
29 ENCLOSED BY ''''  
30 IGNORE 1 ROWS;
```

Below the editor, the 'Output' tab is selected, showing the 'Action Output' table:

#	Time	Action	Message
105128	11:27:36	CREATE TABLE IF NOT EXISTS users (id INT PRIMARY KEY, name VARCHAR(50), surname VARCHAR(50), phone VARCHAR(50), email VARCHAR(200), birth_date VARCHAR(50), country VARCHAR(50), city VARCHAR(100), postal_code VARCHAR(50), address VARCHAR(200), signup_date DATE, user_segment VARCHAR(50), income_band VARCHAR(50))	0 row(s) affected
105129	11:27:36	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__european_users.csv' INTO TABLE users FIELDS TERMINATED BY ',' ENCLOSED BY '''' IGNORE 1 ROWS;	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0
105130	11:27:36	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_users.csv' INTO TABLE users FIELDS TERMINATED BY ',' ENCLOSED BY '''' IGNORE 1 ROWS;	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0

Después, se establecen los constraints de las diferentes tablas de dimensiones que se relacionarán directamente con la tabla de hechos, a modo de Foreign Key. En este caso las tablas de dimensiones son: companies, credit_cards, y users.



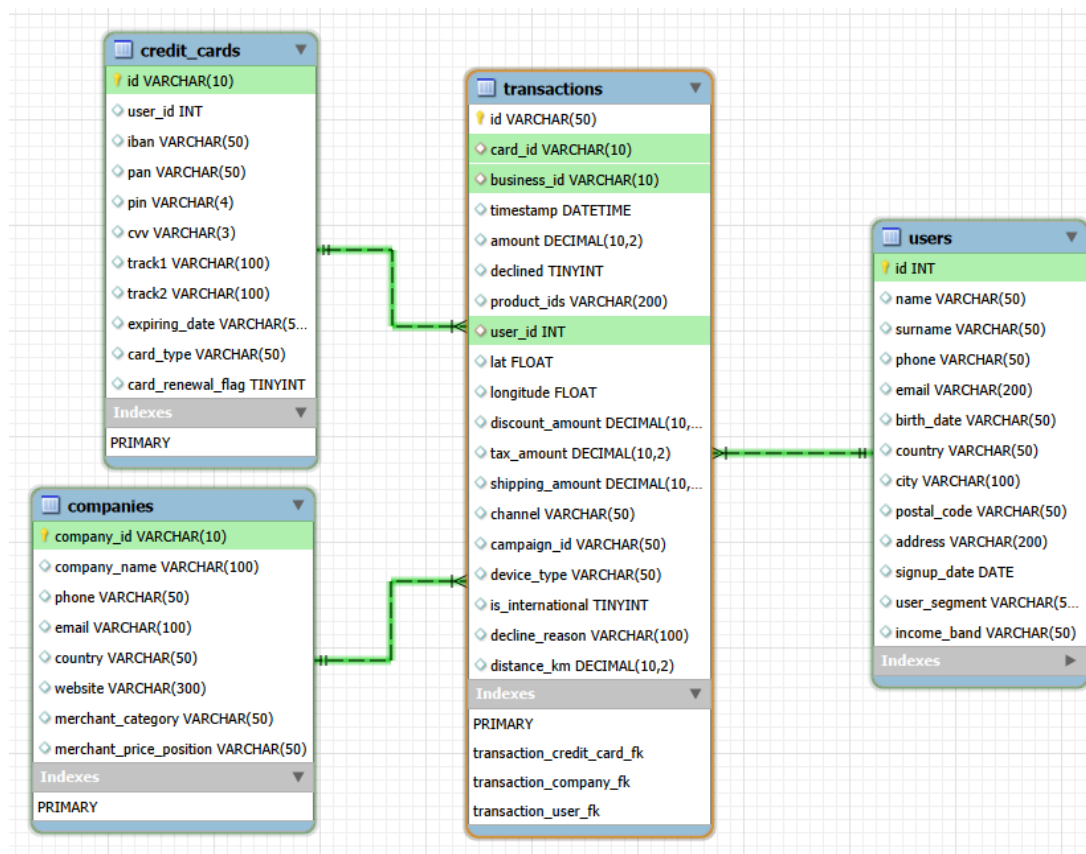
```
company x
1 • ALTER TABLE ex8_transactions.transactions
2   ADD CONSTRAINT transaction_credit_card_fk
3   FOREIGN KEY (card_id)
4   REFERENCES credit_cards(id);
5
6 • ALTER TABLE ex8_transactions.transactions
7   ADD CONSTRAINT transaction_company_fk
8   FOREIGN KEY (business_id)
9   REFERENCES companies(company_id);
10
11 • ALTER TABLE ex8_transactions.transactions
12   ADD CONSTRAINT transaction_user_fk
13   FOREIGN KEY (user_id)
14   REFERENCES users(id);
15
```

Output

Action Output

#	Time	Action	Message
✓ 105131	11:29:45	ALTER TABLE ex8_transactions.transactions ADD CONSTRAINT transaction_credit_card_fk FOREIGN KEY ...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
✓ 105132	11:29:46	ALTER TABLE ex8_transactions.transactions ADD CONSTRAINT transaction_company_fk FOREIGN KEY (b...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
✓ 105133	11:29:48	ALTER TABLE ex8_transactions.transactions ADD CONSTRAINT transaction_user_fk FOREIGN KEY (user_j...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Diagrama resultante.



El sistema se compone de cuatro tablas: transactions, credit_cards, users, y companies.

La tabla companies está compuesta por datos de las empresas, y se compone de las variables: company_id, nombre de la empresa, teléfono, email, país, página web, categoría y posición en el mercado.

La tabla credit cards tiene los datos de las tarjetas de crédito de los usuarios y cuenta con las variables: id, user_id, iban, pan, pin, cvv, track1 y track2, fecha de expiración, tipo de tarjeta y si la tarjeta tiene que renovarse.

La tabla users contiene la información de los usuarios, y está compuesta de las variables: id, nombre, apellido, teléfono, email, fecha de nacimiento, país, ciudad, código postal, dirección, fecha de alta, clase de usuario y nivel de ingresos.

Por otro lado, la tabla transactions contiene los datos de cada transacción realizada. Se compone de las variables: id, card_id, business_id, fecha concreta, importe, si la transacción ha sido rechazada, productos, user_id, latitud, longitud, descuento, impuestos, coste de envío, canal de venta, campaña, tipo de dispositivo, si es una transacción internacional, motivo del rechazo y distancia en kilómetros.

Por todo ello, las tablas companies, credit_cards y users son tablas de dimensiones, ya que describen las empresas, tarjetas de crédito y usuarios como tal, mientras que la tabla transactions es una tabla de hechos, ya que es un registro de operaciones puntuales.

Una empresa, usuario, o tarjeta, puede estar asociada a varias transacciones, mientras que cada transacción está asociada a una sola empresa, usuario y tarjeta, por lo que la relación de cada una de ellas con la tabla transactions será de 1:N.

En cuanto a la granularidad, las tablas cuentan con una fila para cada empresa, tarjeta, usuario, y transacción.

Las tablas cuentan con índices para cada una de sus Primary Keys, y la tabla transactions cuenta adicionalmente con los Foreign Keys que establecen la relación con el resto de tablas, con las variables company_id de companies, user_id de users, y card_id de credit_cards.

A su vez, las claves primarias de cada tabla son company_id en companies, id en credit_cards, id en users, e id en transactions.

Exercici 9

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

Resposta:

The screenshot shows a database management tool interface. At the top, a tab labeled 'company' is active. Below it, a SQL query is entered in a text area:

```
1 • SELECT * FROM ex8_transactions.users AS u
2 WHERE EXISTS (
3     SELECT 1
4     FROM ex8_transactions.transactions AS t
5     WHERE u.id = t.user_id
6     GROUP BY t.user_id
7     HAVING COUNT(*) > 80
8 );
```

Below the query editor, the 'Result Grid' is displayed, showing a table with the following columns: id, name, surname, phone, email, birth_date, country, city, postal_code, address, and signup_date. The table contains four rows of data:

id	name	surname	phone	email	birth_date	country	city	postal_code	address	signup_date
185	Molly	Gilliam	0800 120 8023	donec@outlook.couk	Dec 21, 1993	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 Mi Rd.	2013-08-11
289	Dxwgi	Hwcru	+98-309-8797	dxwgi.hwcru@example.com	Aug 20, 1976	Germany	Stuttgart	70173	82 Hwcru Street	2013-12-11
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	May 3, 1974	Italy	Genoa	16100	53 Astuw Street	2013-05-29
454	Sfzzoh	Xgvfridxs	+58-495-3945	sfzzoh.xgvfridxs@example.com	Aug 28, 1962	Poland	Gdansk	80-001	52 Xgvfridxs Street	2014-08-30

Below the result grid, there is a tab labeled 'users 8'. At the bottom, the 'Output' pane shows the execution log with the following entries:

#	Time	Action	Message
105132	11:29:46	ALTER TABLE ex8_transactions.transactions ADD CONSTRAINT transaction_company_fk FOREIGN KEY (...)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
105133	11:29:48	ALTER TABLE ex8_transactions.transactions ADD CONSTRAINT transaction_user_fk FOREIGN KEY (user_...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
105134	11:33:35	SELECT * FROM ex8_transactions.users AS u WHERE EXISTS (SELECT 1 FROM ex8_transactions.trans...	4 row(s) returned

Exercici 10

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

Resposta:

The screenshot shows a database query editor with a SQL query and its results. The query is as follows:

```
1 • SELECT c.company_name, cc.iban, ROUND(AVG(t.amount),2) AS avg_amount
2 FROM credit_cards AS cc
3 JOIN transactions AS t
4 ON cc.id = t.card_id
5 JOIN companies AS c
6 ON c.company_id = t.business_id
7 WHERE c.company_name = 'Donec Ltd'
8 GROUP BY cc.iban, c.company_name;
```

The results are displayed in a table with the following columns: company_name, iban, and avg_amount. The table contains 8 rows of data for the company 'Donec Ltd'.

company_name	iban	avg_amount
Donec Ltd	CA901355773586771626108990	361.01
Donec Ltd	US729212642816955651428791	142.56
Donec Ltd	US899450803230933276819251	246.90
Donec Ltd	US581097432668325541142297	154.01
Donec Ltd	CA441226267105857473570574	250.51
Donec Ltd	CA332171962352073015086775	203.95
Donec Ltd	US985359628701198880628720	438.16
Donec Ltd	US788111972997693675524914	520.37

The bottom of the screenshot shows the 'Action Output' section with the following messages:

- 105133 11:29:48 ALTER TABLE ex8_transactions.transactions ADD CONSTRAINT transaction_user_fk FOREIGN KEY (user_... 100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
- 105134 11:33:35 SELECT * FROM ex8_transactions.users AS u WHERE EXISTS (SELECT 1 FROM ex8_transactions.trans... 4 row(s) returned
- 105135 11:34:09 SELECT c.company_name, cc.iban, ROUND(AVG(t.amount),2) AS avg_amount FROM credit_cards AS cc J... 371 row(s) returned

Nivell 2

Exercici 1

Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.

Respuesta:

The screenshot shows a database interface with a query editor and a results grid. The query editor contains the following SQL code:

```
1 • SELECT DATE(timestamp) AS transaction_date, SUM(amount) AS total_amount
2 FROM ex8_transactions.transactions
3 WHERE declined = 0
4 GROUP BY transaction_date
5 ORDER BY total_amount DESC
6 LIMIT 5;
```

Below the query editor, the results are displayed in a grid with two columns: `transaction_date` and `total_amount`. The results are sorted by `total_amount` in descending order.

transaction_date	total_amount
2023-02-20	15728.40
2019-11-18	15447.89
2022-12-13	14944.00
2019-03-18	14896.47
2017-12-20	14367.96

At the bottom of the interface, there is an 'Output' section showing a log of database actions. The log includes the following entries:

#	Time	Action	Message
105134	11:33:35	SELECT * FROM ex8_transactions.users AS u WHERE EXISTS (SELECT 1 FROM ex8_transactions.trans...	4 row(s) returned
105135	11:34:09	SELECT c.company_name, cc.iban, ROUND(AVG(t.amount),2) AS avg_amount FROM credit_cards AS cc J...	371 row(s) returned
105136	11:34:45	SELECT DATE(timestamp) AS transaction_date, SUM(amount) AS total_amount FROM ex8_transactions.trans...	5 row(s) returned

Exercici 2

Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.

Respuesta:

The screenshot shows a SQL IDE interface with a query editor and a results grid.

Query Editor:

```
1 • SELECT c.company_name, c.phone, c.country, DATE(t.timestamp) AS transaction_date, t.amount
2 FROM ex8_transactions.companies AS c
3 JOIN ex8_transactions.transactions AS t
4 ON c.company_id = t.business_id
5 WHERE t.declined = 0
6 AND t.amount BETWEEN 350 AND 400
7 AND DATE(t.timestamp) IN ('2015-04-29', '2018-07-20', '2024-03-13')
8 ORDER BY t.amount DESC;
```

Result Grid:

company_name	phone	country	transaction_date	amount
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29	369.20
Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20	358.64
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13	352.21
Tempor Diam Institute	+89 848 141 3136	Netherlands	2024-03-13	350.71

Result 11:

Output:

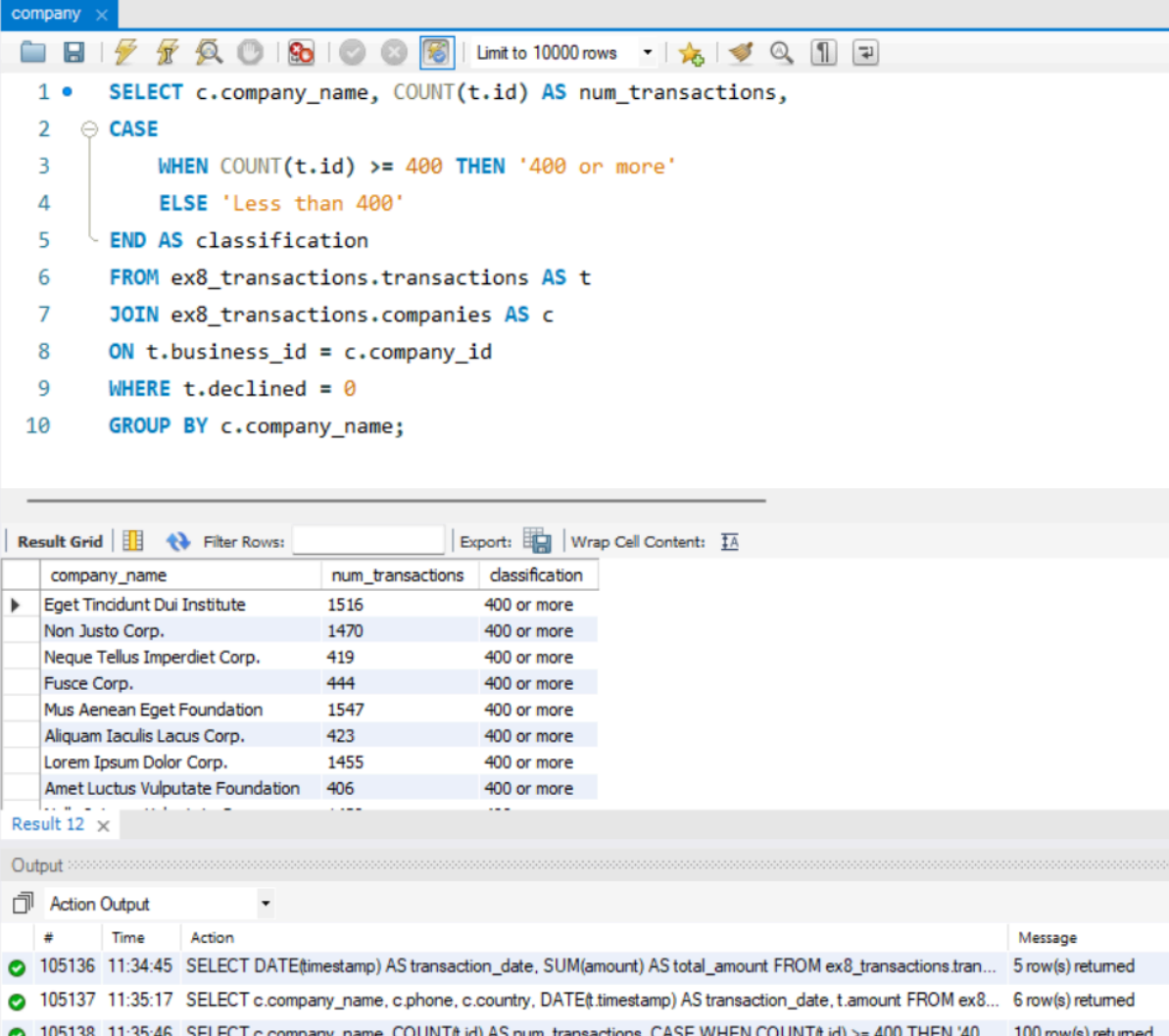
Action Output:

#	Time	Action	Message
✓ 105135	11:34:09	SELECT c.company_name, cc.iban, ROUND(AVG(t.amount),2) AS avg_amount FROM credit_cards AS cc J...	371 row(s) returned
✓ 105136	11:34:45	SELECT DATE(timestamp) AS transaction_date, SUM(amount) AS total_amount FROM ex8_transactions.tran...	5 row(s) returned
✓ 105137	11:35:17	SELECT c.company_name, c.phone, c.country, DATE(t.timestamp) AS transaction_date, t.amount FROM ex8...	6 row(s) returned

Exercici 3

Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis si tenen igual o més de 400 transaccions o menys.

Resposta:



The screenshot shows a SQL IDE interface with a query editor and a results pane. The query is as follows:

```
1 • SELECT c.company_name, COUNT(t.id) AS num_transactions,
2 CASE
3     WHEN COUNT(t.id) >= 400 THEN '400 or more'
4     ELSE 'Less than 400'
5 END AS classification
6 FROM ex8_transactions.transactions AS t
7 JOIN ex8_transactions.companies AS c
8 ON t.business_id = c.company_id
9 WHERE t.declined = 0
10 GROUP BY c.company_name;
```

The results pane displays a table with the following data:

company_name	num_transactions	classification
Eget Tincidunt Dui Institute	1516	400 or more
Non Justo Corp.	1470	400 or more
Neque Tellus Imperdiet Corp.	419	400 or more
Fusce Corp.	444	400 or more
Mus Aenean Eget Foundation	1547	400 or more
Aliquam Iaculis Lacus Corp.	423	400 or more
Lorem Ipsum Dolor Corp.	1455	400 or more
Amet Luctus Vulputate Foundation	406	400 or more

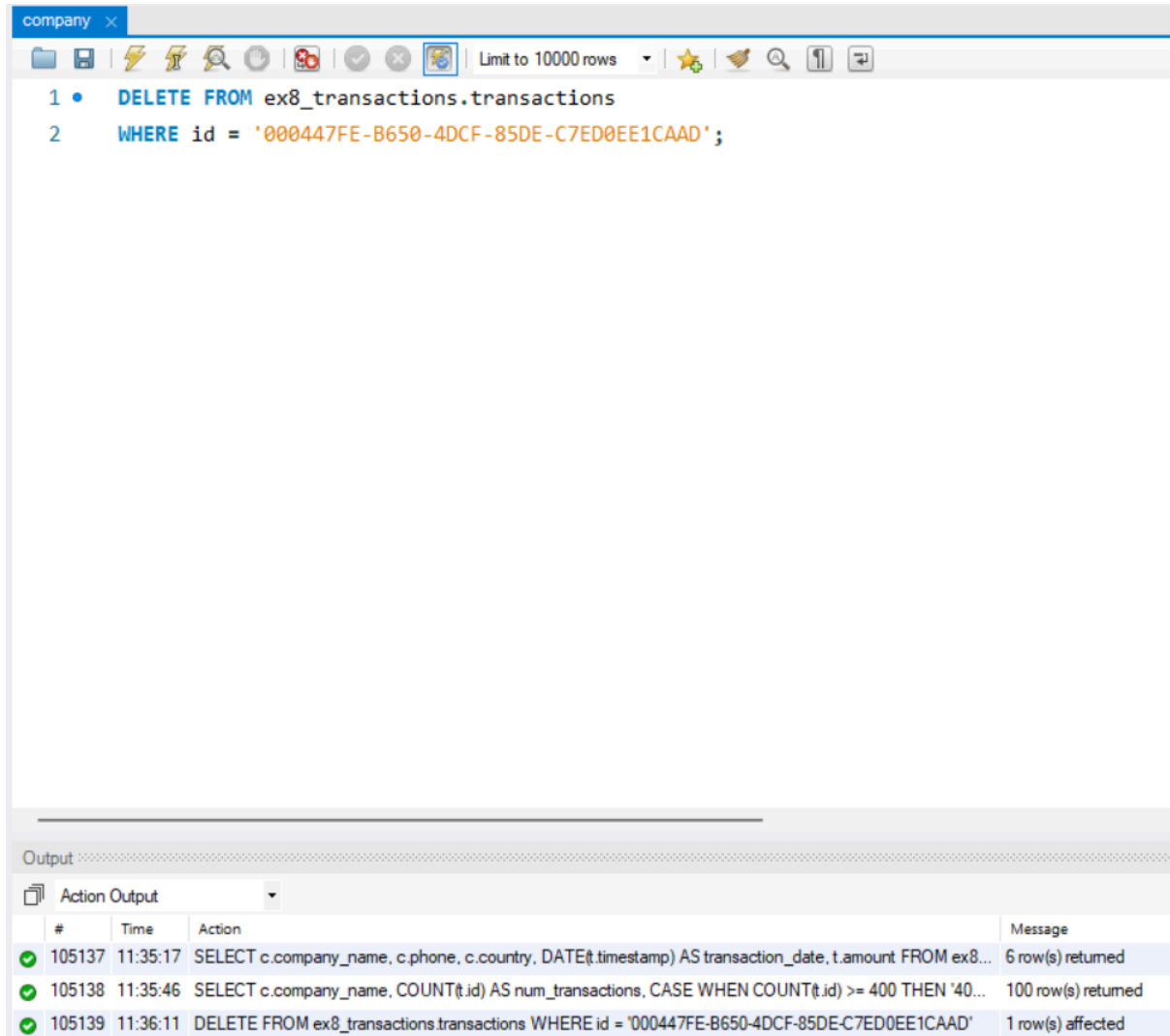
The output pane shows the execution log with the following entries:

#	Time	Action	Message
105136	11:34:45	SELECT DATE(timestamp) AS transaction_date, SUM(amount) AS total_amount FROM ex8_transactions.tran...	5 row(s) returned
105137	11:35:17	SELECT c.company_name, c.phone, c.country, DATE(timestamp) AS transaction_date, t.amount FROM ex8...	6 row(s) returned
105138	11:35:46	SELECT c.company_name, COUNT(t.id) AS num_transactions, CASE WHEN COUNT(t.id) >= 400 THEN '40...	100 row(s) returned

Exercici 4

Elimina de la taula transaction el registre amb ID
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades

Resposta:



The screenshot shows a database management interface with a query editor and an output window. The query editor contains the following SQL statement:

```
1 • DELETE FROM ex8_transactions.transactions
2 WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

The output window, titled "Output", shows the "Action Output" tab. It displays a log of database actions with the following columns: #, Time, Action, and Message.

#	Time	Action	Message
✓ 105137	11:35:17	SELECT c.company_name, c.phone, c.country, DATE(t.timestamp) AS transaction_date, t.amount FROM ex8...	6 row(s) returned
✓ 105138	11:35:46	SELECT c.company_name, COUNT(t.id) AS num_transactions, CASE WHEN COUNT(t.id) >= 400 THEN '40...	100 row(s) returned
✓ 105139	11:36:11	DELETE FROM ex8_transactions.transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected

Exercici 5

La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.

Respuesta:

The screenshot displays a database management interface. At the top, a tab labeled 'company' is active. Below the toolbar, a SQL script is entered in a text area. The script consists of two parts: a 'CREATE VIEW' statement and a 'SELECT' statement. The 'CREATE VIEW' statement defines a view named 'VistaMarketing' based on a join of 'ex8_transactions.companies' and 'ex8_transactions.transactions'. It filters for 't.declined = 0' and calculates the average transaction amount per company. The 'SELECT' statement queries the 'VistaMarketing' view, ordering the results by 'avg_amount' in descending order.

```
1 • CREATE VIEW VistaMarketing AS
2 SELECT c.company_name, c.phone, c.country, ROUND(AVG(t.amount),2) AS avg_amount
3 FROM ex8_transactions.companies AS c
4 JOIN ex8_transactions.transactions AS t
5 ON c.company_id = t.business_id
6 WHERE t.declined = 0
7 GROUP BY c.company_name, c.phone, c.country;
8
9 SELECT * FROM VistaMarketing
10 ORDER BY avg_amount DESC;
```

Below the SQL editor, the 'Result Grid' is visible, showing the output of the SQL execution. The grid has four columns: 'company_name', 'phone', 'country', and 'avg_amount'. The data is sorted by 'avg_amount' in descending order.

company_name	phone	country	avg_amount
Ac Fermentum Incorporated	+64 601 244 6320	Germany	330.71
Ut Semper Foundation	+10 283 500 6449	Sweden	307.30
Dolor Vitae Limited	+6 421 567 9195	France	306.21
Orci Adipiscing Limited	+60 897 390 5820	United Kingdom	305.87
Nunc In Foundation	+13 267 741 2619	Italy	305.67
Maecenas Malesuada Fringilla Inc.	+51 673 540 5289	Netherlands	304.49
Turpis Company	+17 624 311 7168	Netherlands	303.47
Nulla Integer Vulputate Corp.	+75 236 687 9162	Sweden	303.28

Below the result grid, the 'Output' section shows the execution log. It contains three entries:

#	Time	Action	Message
105140	11:37:14	CREATE VIEW VistaMarketing	Error Code: 1064. You have
105141	11:38:08	CREATE VIEW VistaMarketing AS SELECT c.company_name, c.phone, c.country, ROUND(AVG(t.amount),2...	0 row(s) affected
105142	11:38:08	SELECT * FROM VistaMarketing ORDER BY avg_amount DESC LIMIT 0, 10000	100 row(s) returned

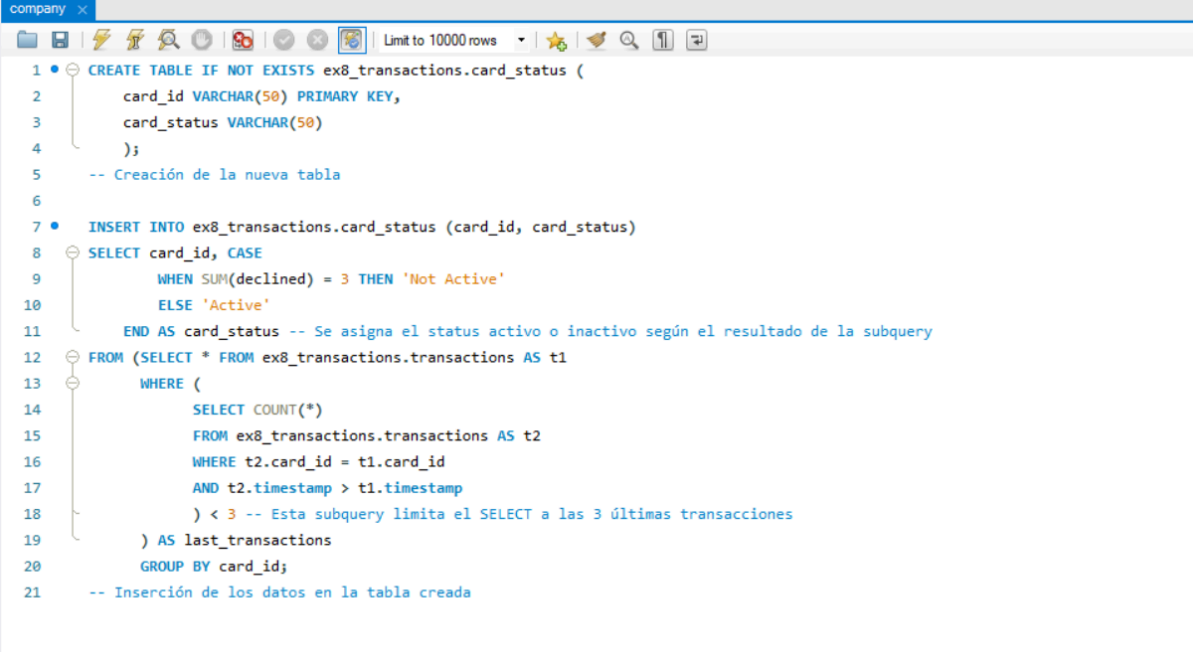
Nivell 3

Exercici 1

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

👉 Quantes targetes estan actives?

Resposta:



```
1 CREATE TABLE IF NOT EXISTS ex8_transactions.card_status (  
2     card_id VARCHAR(50) PRIMARY KEY,  
3     card_status VARCHAR(50)  
4 );  
5 -- Creación de la nueva tabla  
6  
7 INSERT INTO ex8_transactions.card_status (card_id, card_status)  
8 SELECT card_id, CASE  
9     WHEN SUM(declined) = 3 THEN 'Not Active'  
10    ELSE 'Active'  
11 END AS card_status -- Se asigna el status activo o inactivo según el resultado de la subquery  
12 FROM (SELECT * FROM ex8_transactions.transactions AS t1  
13     WHERE (  
14         SELECT COUNT(*)  
15         FROM ex8_transactions.transactions AS t2  
16         WHERE t2.card_id = t1.card_id  
17         AND t2.timestamp > t1.timestamp  
18     ) < 3 -- Esta subquery limita el SELECT a las 3 últimas transacciones  
19 ) AS last_transactions  
20 GROUP BY card_id;  
21 -- Inserción de los datos en la tabla creada
```

Output

#	Time	Action	Message
105142	11:38:08	SELECT * FROM VistaMarketing ORDER BY avg_amount DESC LIMIT 0, 10000	100 row(s) returned
105143	11:39:53	CREATE TABLE IF NOT EXISTS ex8_transactions.card_status (card_id VARCHAR(50) PRIMARY KEY, c...	0 row(s) affected
105144	11:39:54	INSERT INTO ex8_transactions.card_status (card_id, card_status) SELECT card_id, CASE WHEN SUM(decl...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

company

Limit to 10000 rows

```

1 • SELECT COUNT(*) AS num_active_cards
2   FROM ex8_transactions.card_status
3   WHERE card_status = 'Active';

```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	num_active_cards
▶	5000

Result 14

Read Only

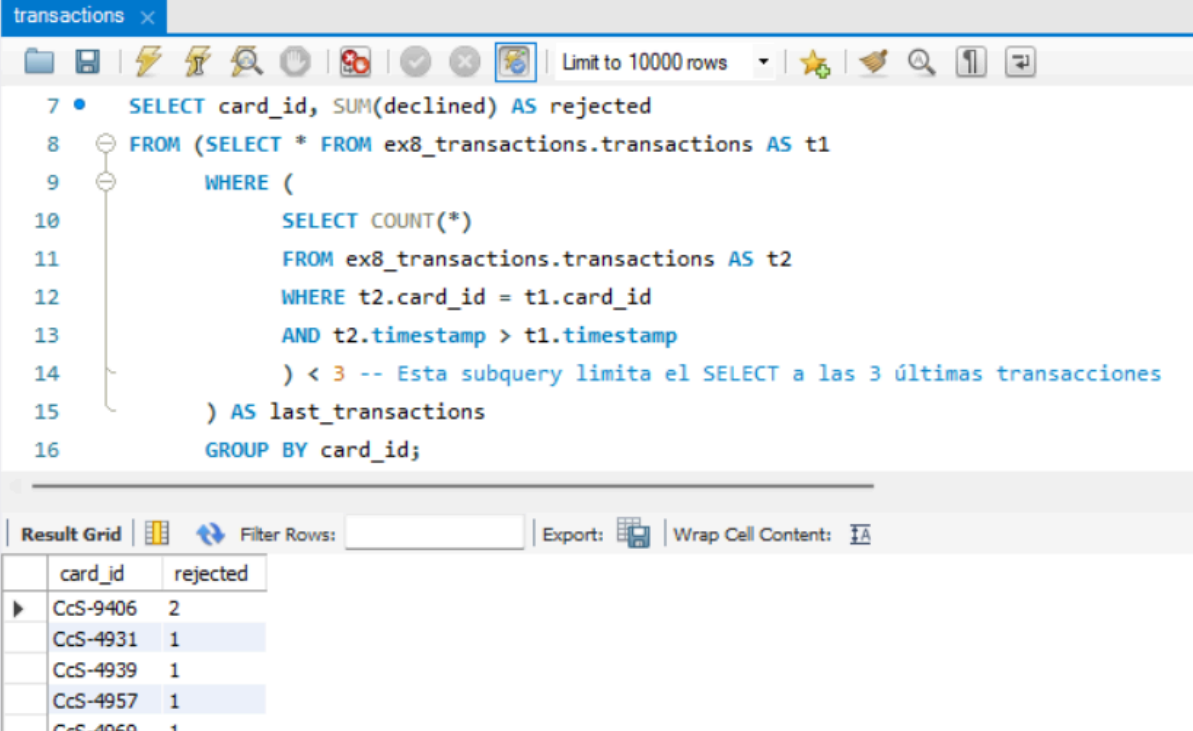
Output

Action Output

#	Time	Action	Message
105143	11:39:53	CREATE TABLE IF NOT EXISTS ex8_transactions.card_status (card_id VARCHAR(50) PRIMARY KEY, c...	0 row(s) affected
105144	11:39:54	INSERT INTO ex8_transactions.card_status (card_id, card_status) SELECT card_id, CASE WHEN SUM(decl...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
105145	11:40:26	SELECT COUNT(*) AS num_active_cards FROM ex8_transactions.card_status WHERE card_status = 'Activ...	1 row(s) returned

Para poder mostrar de mejor manera el proceso de resolución del ejercicio, este es un desglose por pasos de los elementos que componen el código final:

Paso 1: Se crea un filtro que tenga en cuenta solamente las últimas 3 transacciones.



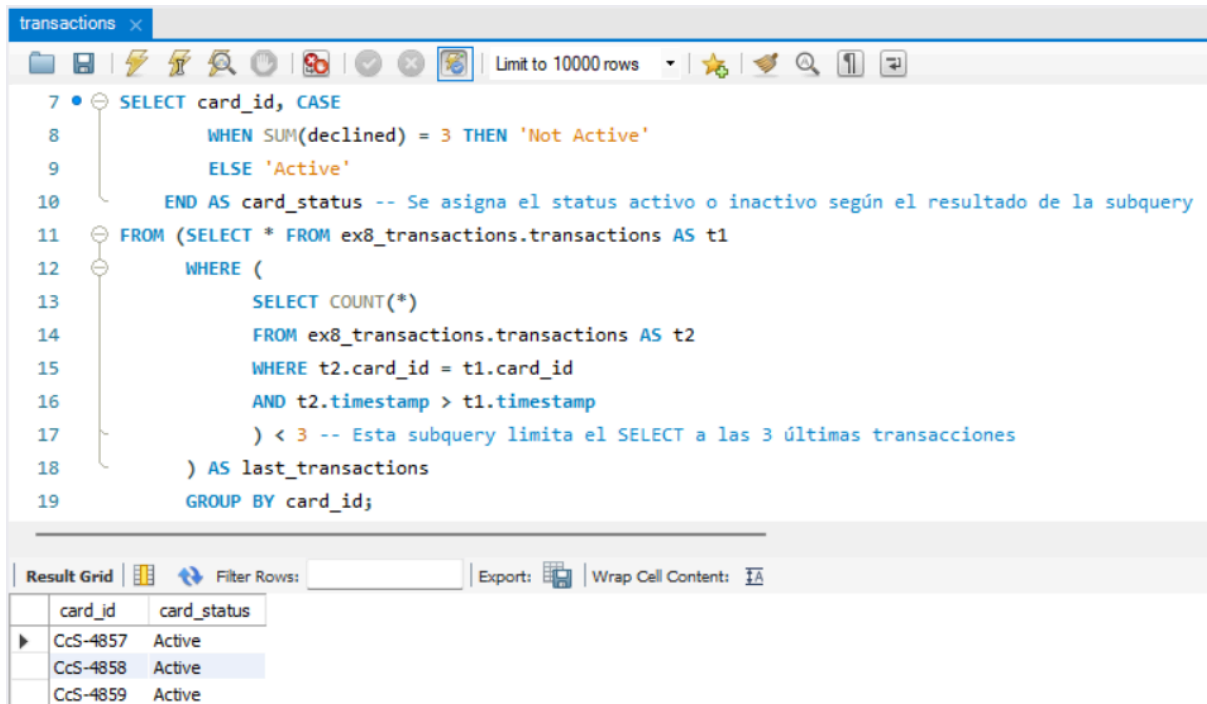
The screenshot shows a SQL IDE window titled "transactions". The query editor contains the following SQL code:

```
7 • SELECT card_id, SUM(declined) AS rejected
8 FROM (SELECT * FROM ex8_transactions.transactions AS t1
9 WHERE (
10     SELECT COUNT(*)
11     FROM ex8_transactions.transactions AS t2
12     WHERE t2.card_id = t1.card_id
13     AND t2.timestamp > t1.timestamp
14 ) < 3 -- Esta subquery limita el SELECT a las 3 últimas transacciones
15 ) AS last_transactions
16 GROUP BY card_id;
```

Below the query editor, the "Result Grid" tab is active, displaying the results of the query. The grid has two columns: "card_id" and "rejected". The results are as follows:

card_id	rejected
CcS-9406	2
CcS-4931	1
CcS-4939	1
CcS-4957	1
CcS-4969	1

Paso 2: Se asigna el status activo o inactivo usando el resultado del SUM(declined) junto con el filtrado anterior.



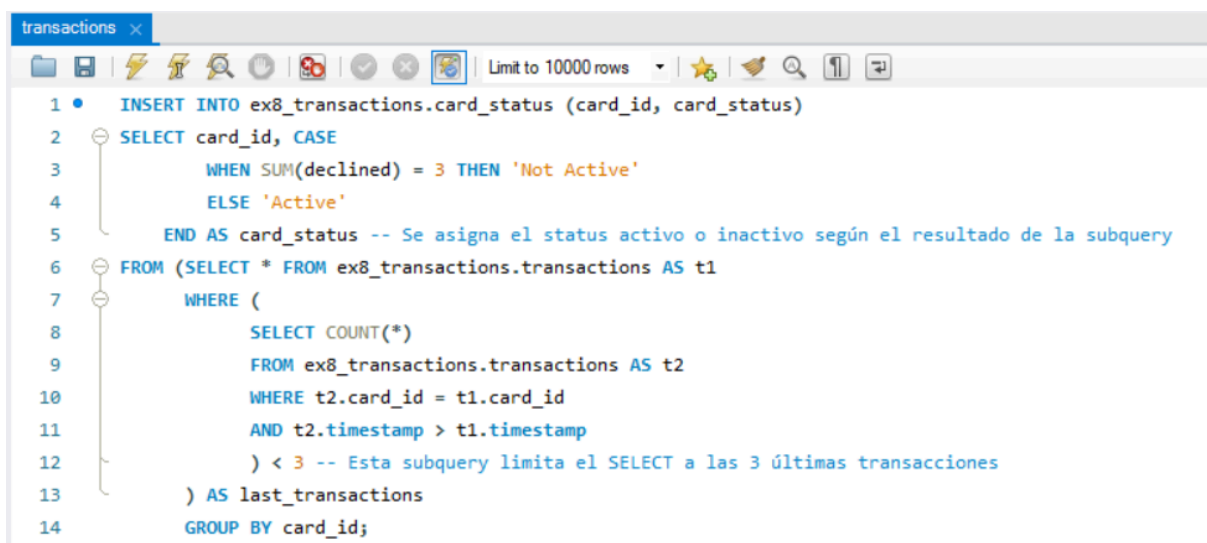
The screenshot shows a SQL IDE window titled 'transactions'. The query editor contains the following SQL code:

```
7 • SELECT card_id, CASE
8     WHEN SUM(declined) = 3 THEN 'Not Active'
9     ELSE 'Active'
10  END AS card_status -- Se asigna el status activo o inactivo según el resultado de la subquery
11  FROM (SELECT * FROM ex8_transactions.transactions AS t1
12        WHERE (
13            SELECT COUNT(*)
14            FROM ex8_transactions.transactions AS t2
15            WHERE t2.card_id = t1.card_id
16            AND t2.timestamp > t1.timestamp
17            ) < 3 -- Esta subquery limita el SELECT a las 3 últimas transacciones
18        ) AS last_transactions
19  GROUP BY card_id;
```

Below the query editor, the 'Result Grid' tab is active, displaying the following data:

card_id	card_status
CcS-4857	Active
CcS-4858	Active
CcS-4859	Active

Paso 3: Se insertan las columnas resultantes en la tabla creada.



The screenshot shows a SQL IDE window titled 'transactions'. The query editor contains the following SQL code:

```
1 • INSERT INTO ex8_transactions.card_status (card_id, card_status)
2  SELECT card_id, CASE
3      WHEN SUM(declined) = 3 THEN 'Not Active'
4      ELSE 'Active'
5  END AS card_status -- Se asigna el status activo o inactivo según el resultado de la subquery
6  FROM (SELECT * FROM ex8_transactions.transactions AS t1
7        WHERE (
8            SELECT COUNT(*)
9            FROM ex8_transactions.transactions AS t2
10           WHERE t2.card_id = t1.card_id
11           AND t2.timestamp > t1.timestamp
12           ) < 3 -- Esta subquery limita el SELECT a las 3 últimas transacciones
13        ) AS last_transactions
14  GROUP BY card_id;
```

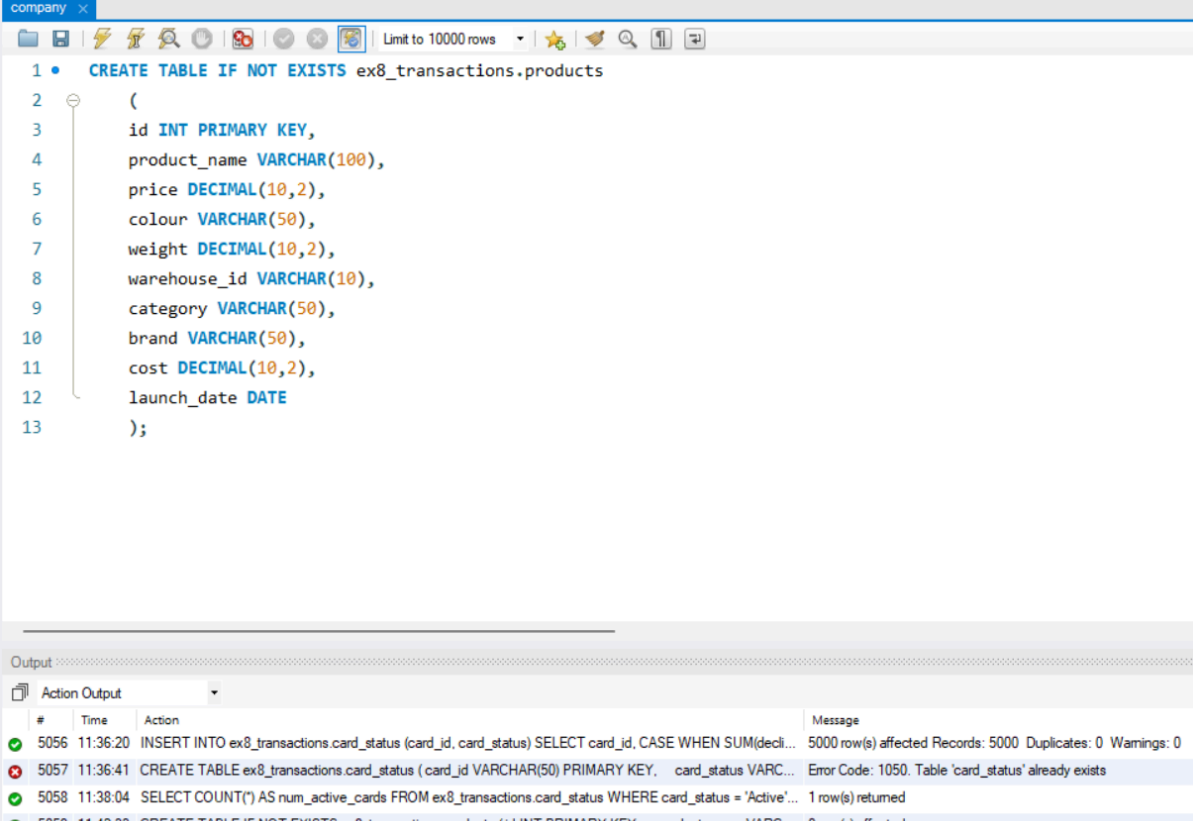
Exercici 2

Crea una taula amb la qual puguem unir les dades de l'arxiu de products.csv amb la base de dades creada (ja que fins ara no podem fer-ho), tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

👉 Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

Respuesta:

Creación de la tabla products.



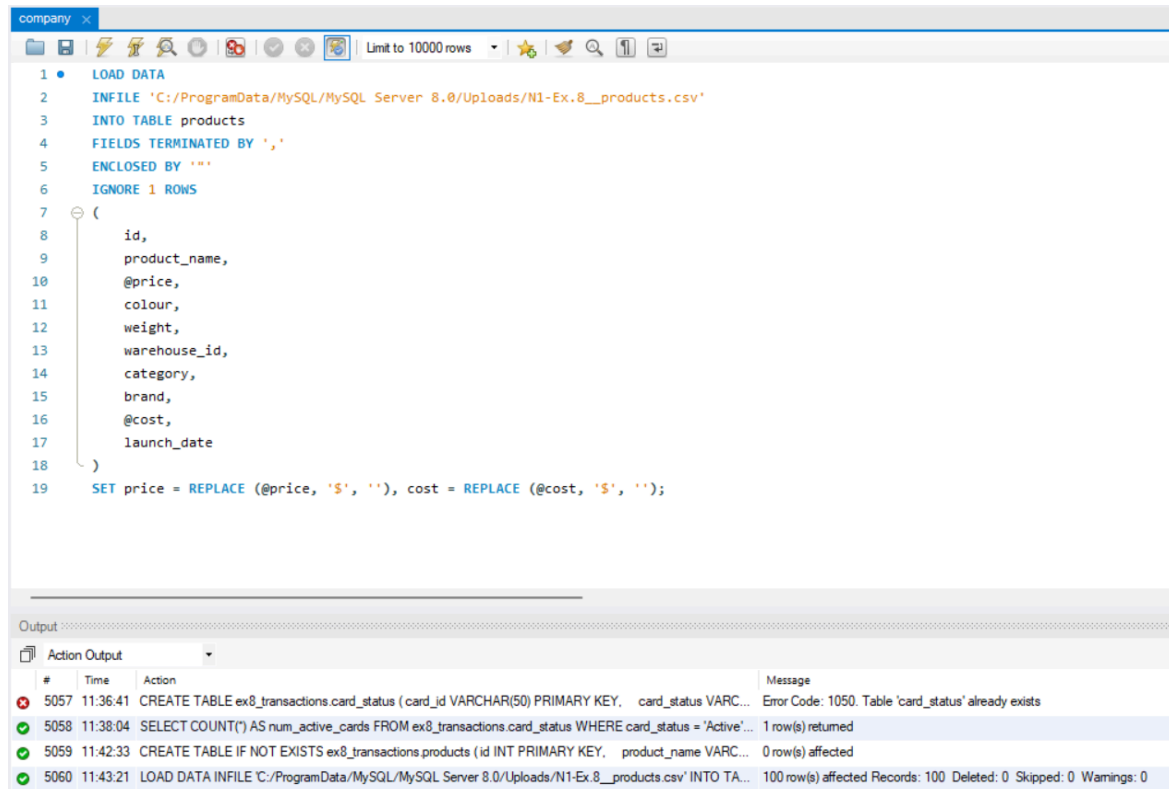
The screenshot shows a database management interface with a query editor and an output window. The query editor contains the following SQL code:

```
1 • CREATE TABLE IF NOT EXISTS ex8_transactions.products
2 (
3     id INT PRIMARY KEY,
4     product_name VARCHAR(100),
5     price DECIMAL(10,2),
6     colour VARCHAR(50),
7     weight DECIMAL(10,2),
8     warehouse_id VARCHAR(10),
9     category VARCHAR(50),
10    brand VARCHAR(50),
11    cost DECIMAL(10,2),
12    launch_date DATE
13 );
```

The output window shows the following results:

#	Time	Action	Message
5056	11:36:20	INSERT INTO ex8_transactions.card_status (card_id, card_status) SELECT card_id, CASE WHEN SUM(decli...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
5057	11:36:41	CREATE TABLE ex8_transactions.card_status (card_id VARCHAR(50) PRIMARY KEY, card_status VARC...	Error Code: 1050. Table 'card_status' already exists
5058	11:38:04	SELECT COUNT(*) AS num_active_cards FROM ex8_transactions.card_status WHERE card_status = 'Active'...	1 row(s) returned
5059	11:42:33	CREATE TABLE IF NOT EXISTS ex8_transactions.products (id INT PRIMARY KEY, product_name VARC...	0 row(s) affected

Se carga el .csv, y se crean variables temporales con @ para eliminar el símbolo '\$' de las variables price y cost, y poder importar el archivo sin problemas.



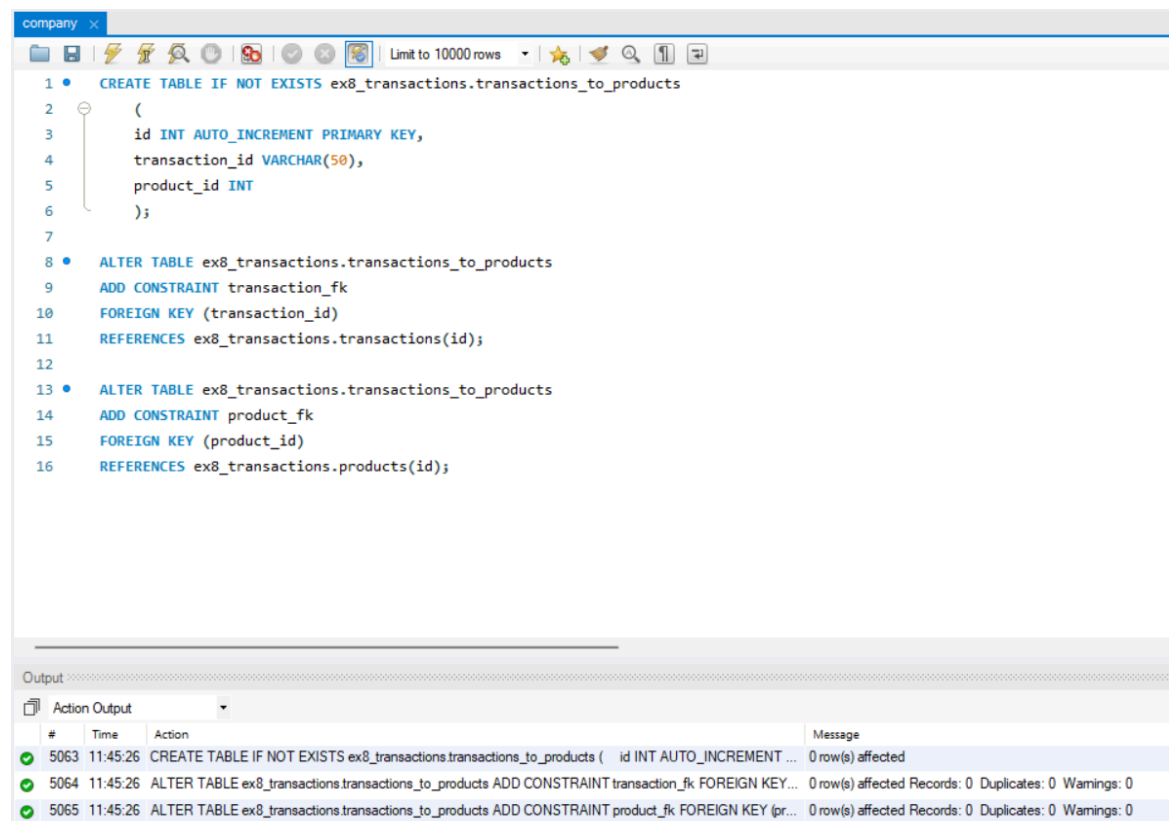
The screenshot shows a MySQL Workbench window with a tab labeled 'company'. The SQL editor contains the following script:

```
1 • LOAD DATA
2   INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__products.csv'
3   INTO TABLE products
4   FIELDS TERMINATED BY ','
5   ENCLOSED BY '"'
6   IGNORE 1 ROWS
7   (
8     id,
9     product_name,
10    @price,
11    colour,
12    weight,
13    warehouse_id,
14    category,
15    brand,
16    @cost,
17    launch_date
18  )
19  SET price = REPLACE (@price, '$', ''), cost = REPLACE (@cost, '$', '');
```

The Output window at the bottom shows the following messages:

#	Time	Action	Message
5057	11:36:41	CREATE TABLE ex8_transactions.card_status (card_id VARCHAR(50) PRIMARY KEY, card_status VARC...	Error Code: 1050. Table 'card_status' already exists
5058	11:38:04	SELECT COUNT(*) AS num_active_cards FROM ex8_transactions.card_status WHERE card_status = 'Active'...	1 row(s) returned
5059	11:42:33	CREATE TABLE IF NOT EXISTS ex8_transactions.products (id INT PRIMARY KEY, product_name VARC...	0 row(s) affected
5060	11:43:21	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__products.csv' INTO TA...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Creación de una tabla que permita relacionar la tabla products con la variable product_ids de la tabla transactions.



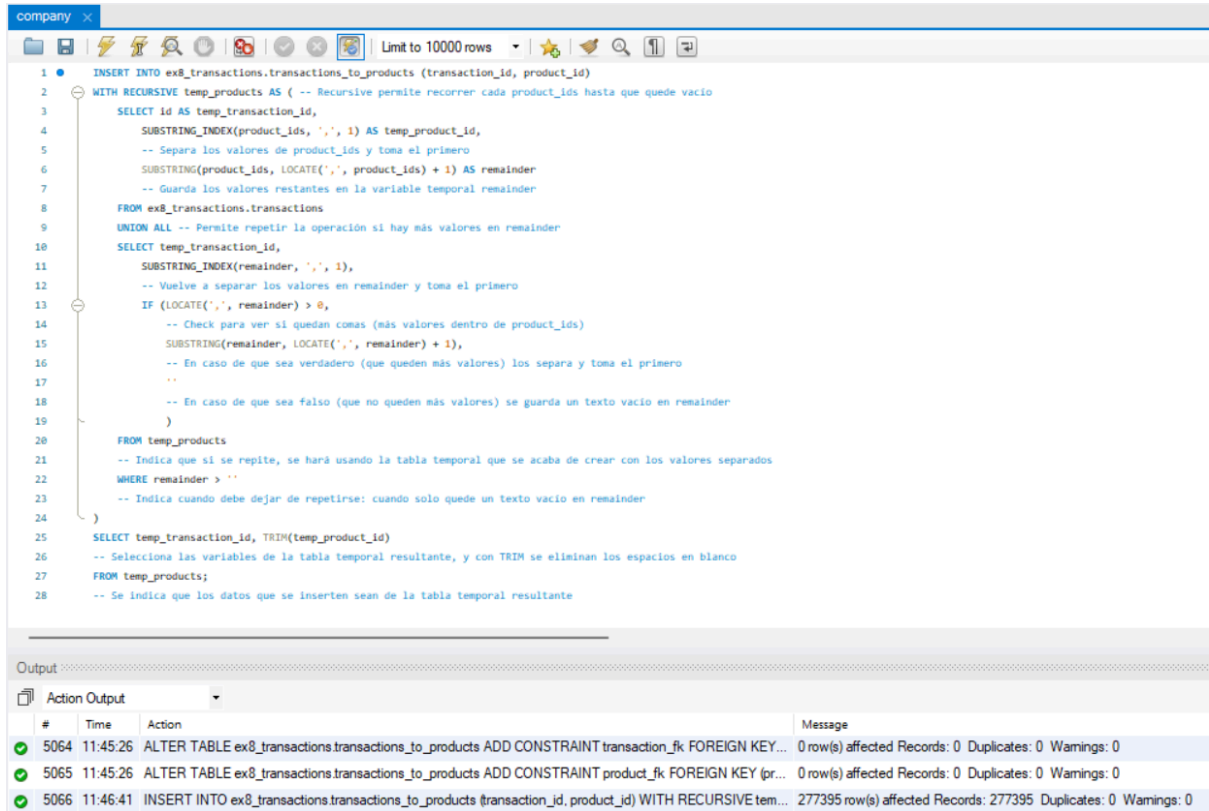
The screenshot shows a MySQL Workbench window with a tab labeled 'company'. The SQL editor contains the following script:

```
1 • CREATE TABLE IF NOT EXISTS ex8_transactions.transactions_to_products
2   (
3     id INT AUTO_INCREMENT PRIMARY KEY,
4     transaction_id VARCHAR(50),
5     product_id INT
6   );
7
8 • ALTER TABLE ex8_transactions.transactions_to_products
9   ADD CONSTRAINT transaction_fk
10  FOREIGN KEY (transaction_id)
11  REFERENCES ex8_transactions.transactions(id);
12
13 • ALTER TABLE ex8_transactions.transactions_to_products
14   ADD CONSTRAINT product_fk
15   FOREIGN KEY (product_id)
16   REFERENCES ex8_transactions.products(id);
```

The Output window at the bottom shows the following messages:

#	Time	Action	Message
5063	11:45:26	CREATE TABLE IF NOT EXISTS ex8_transactions.transactions_to_products (id INT AUTO_INCREMENT ...	0 row(s) affected
5064	11:45:26	ALTER TABLE ex8_transactions.transactions_to_products ADD CONSTRAINT transaction_fk FOREIGN KEY...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
5065	11:45:26	ALTER TABLE ex8_transactions.transactions_to_products ADD CONSTRAINT product_fk FOREIGN KEY (pr...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Inserción de datos en la nueva tabla. Para ello se deben realizar una serie de modificaciones para poder separar los diferentes valores que puede haber guardadas en una sola línea de la variable product_ids.



The screenshot shows a SQL IDE window titled 'company'. The main editor displays a SQL script with 28 lines of code. The script uses a recursive common table expression (WITH RECURSIVE) to process 'product_ids' from a table named 'ex8_transactions.transactions'. It splits the IDs into individual product IDs and inserts them into a new table 'ex8_transactions.transactions_to_products'. The script includes comments in Spanish explaining the recursive process and the use of string functions like SUBSTRING_INDEX and LOCATE. The output pane at the bottom shows the execution results of three SQL actions.

```
1 INSERT INTO ex8_transactions.transactions_to_products (transaction_id, product_id)
2 WITH RECURSIVE temp_products AS ( -- Recursive permite recorrer cada product_ids hasta que quede vacío
3     SELECT id AS temp_transaction_id,
4           SUBSTRING_INDEX(product_ids, ',', 1) AS temp_product_id,
5           -- Separa los valores de product_ids y toma el primero
6           SUBSTRING(product_ids, LOCATE(',', product_ids) + 1) AS remainder
7           -- Guarda los valores restantes en la variable temporal remainder
8     FROM ex8_transactions.transactions
9     UNION ALL -- Permite repetir la operación si hay más valores en remainder
10    SELECT temp_transaction_id,
11          SUBSTRING_INDEX(remainder, ',', 1),
12          -- Vuelve a separar los valores en remainder y toma el primero
13          IF (LOCATE(',', remainder) > 0,
14             -- Check para ver si quedan comas (más valores dentro de product_ids)
15             SUBSTRING(remainder, LOCATE(',', remainder) + 1),
16             -- En caso de que sea verdadero (que queden más valores) los separa y toma el primero
17             ''
18             -- En caso de que sea falso (que no queden más valores) se guarda un texto vacío en remainder
19          )
20     FROM temp_products
21     -- Indica que si se repite, se hará usando la tabla temporal que se acaba de crear con los valores separados
22     WHERE remainder > ''
23     -- Indica cuando debe dejar de repetirse: cuando solo quede un texto vacío en remainder
24 )
25 SELECT temp_transaction_id, TRIM(temp_product_id)
26 -- Selecciona las variables de la tabla temporal resultante, y con TRIM se eliminan los espacios en blanco
27 FROM temp_products;
28 -- Se indica que los datos que se inserten sean de la tabla temporal resultante
```

Output

#	Time	Action	Message
5064	11:45:26	ALTER TABLE ex8_transactions.transactions_to_products ADD CONSTRAINT transaction_fk FOREIGN KEY...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
5065	11:45:26	ALTER TABLE ex8_transactions.transactions_to_products ADD CONSTRAINT product_fk FOREIGN KEY (pr...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
5066	11:46:41	INSERT INTO ex8_transactions.transactions_to_products (transaction_id, product_id) WITH RECURSIVE tem...	277395 row(s) affected Records: 277395 Duplicates: 0 Warnings: 0

Revisión P2P

Revisión hecha por: Sònia Shah

En esta sección se va a indicar qué comentarios y recomendaciones se han recibido en la revisión peer to peer para cada ejercicio. En caso de no aplicar un cambio a pesar de haber recibido una recomendación se justificará el porqué.

Nivell 1

Exercici 1

Se ha indicado que antes de crear una nueva base de datos hay que asegurarse de que no haya una ya con ese nombre, por lo que se tiene que comenzar siempre con DROP DATABASE IF EXISTS. Así en caso de ejecutar el código en otro ordenador se borrará una posible base de datos ya existente con ese nombre, y se creará la nueva base de datos sin problema.

He aplicado este cambio.

Exercici 2

Se ha comentado que no hace falta poner varias variables en el GROUP BY, y que solamente con el Primary Key (id) ya funciona, a pesar de que en el curso de Fonaments se nos enseñó lo contrario.

He decidido no aplicar este cambio ya que el ejercicio funciona de igual manera tal como está, pero es una indicación que tendré en cuenta de aquí en adelante.

Exercici 3

Se ha realizado el mismo comentario para los tres apartados de este ejercicio: para realizar una subconsulta correlacionada sería mejor utilizar EXISTS o NOT EXISTS en lugar de WHERE ... IN o WHERE ... NOT IN, ya que se trata de una buena práctica.

He aplicado estos cambios.

Exercici 4

Se ha indicado que utilizar USE transactions; al principio de este ejercicio es redundante, ya que se ha utilizado al principio del código, en el ejercicio 1, y no se ha cambiado en ningún momento.

He aplicado este cambio.

Exercici 5

No se han recibido comentarios en este ejercicio.

Exercici 6

No se han recibido comentarios en este ejercicio.

Exercici 7

No se han recibido comentarios en este ejercicio.

Exercici 8

Se ha vuelto a indicar el uso de DROP DATABASE IF EXISTS antes de crear una nueva base de datos.

He aplicado este cambio.

También se ha indicado que a la hora de definir el tipo de variable de las columnas de las tablas, puede ser recomendable utilizar VARCHAR en todas ellas, en lugar de otros como DECIMAL o FLOAT, para asegurarse de que sea siempre compatible con el resto de tablas y no haya disonancia en el tipo de variable.

He decidido no aplicar este cambio, ya que al haber definido yo el tipo de variable de todas las tablas en este ejercicio, me he asegurado de que coincidieran para no tener errores, pero es una recomendación que tendré en cuenta a futuro.

Exercici 9

Se ha vuelto a indicar el uso de EXISTS en lugar de WHERE ... IN.

He aplicado este cambio.

Exercici 10

Se ha indicado que no es necesario el uso de LEFT JOIN, ya que solamente con JOIN el código funciona correctamente.

He aplicado este cambio.

Nivell 2

Exercici 1

Se ha señalado la necesidad de añadir la condición `declined = 0`, ya que afecta directamente al resultado de la ejecución del código.

He aplicado este cambio.

Exercici 2

Se ha vuelto a indicar la necesidad de añadir la condición `declined = 0`.

Se ha indicado también que es preferible utilizar `BETWEEN` en lugar de `WHERE ... > ... AND ... < ...`.

He aplicado estos cambios.

Exercici 3

Se ha vuelto a indicar la necesidad de añadir la condición `declined = 0`.

He aplicado este cambio.

Exercici 4

No se han recibido comentarios en este ejercicio.

Exercici 5

Se ha vuelto a indicar la necesidad de añadir la condición `declined = 0`.

He aplicado este cambio.

Nivell 3

Exercici 1

Se ha indicado que se puede realizar el ejercicio de otra manera utilizando el código ROW NUMBER y PARTITION P2P, en lugar de la manera que he escrito yo.

He decidido no aplicar este cambio, ya que al no haber utilizado nunca estas funciones y cláusulas, tuve que encontrar otra manera de resolverlo, y el código resultante funciona igualmente.

Exercici 2

Se ha recomendado utilizar JOIN JSON_TABLE para resolver este ejercicio, ya que acorta mucho el código, lo cual es recomendable.

He decidido no aplicar este cambio, ya que para resolverlo estuve buscando una manera de hacerlo que entendiera mejor, aunque tuviera que utilizar más pasos, de manera que pudiera aprender la lógica detrás de este proceso a la vez que resolviera el ejercicio. Para ello utilicé WITH RECURSIVE y SUBSTRING_INDEX, que no conocía anteriormente, pero que supe entender mejor cómo utilizar.